14 haftalık yol haritasında 7. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

HAFTA 07

Sıralamanın Ötesi

Öbek Sıralaması, Doğrusal Zamanlı Sıralama ve Ara Proje

MODÜL II · ARAMA VE SIRALAMA · H4–H7

HAFTANIN BÜYÜK SORUSU

Karşılaştırma yapmadan sıralamak mümkün mü ve Ω(n log n) sınırı nasıl "aşılır"?

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 4
1.1 Öğrenme çıktıları 4
2. Geçen Haftadan ve AP-1'den Bağlantı 5
3. Kavramsal Anlatım: Dizideki Ağaç, Öncelik ve Alt Sınırın Kapsamı 6
3.1 Tam ikili ağaç ve dizi gösterimi 6
3.2 Öbek özelliği ve öncelik kuyruğu 7
3.3 Ω(n log n) alt sınırı ve “aşılması”nın koşulları 8
3.4 Kaputun Altında: önbellek ve sayaç dizisinin belleği 8
4. Algoritmalar 9
4.1 Öbeğe ekleme ve en büyüğü çıkarma (yukarı ve aşağı süzme) 9
4.2 Aşağıdan yukarıya öbek kurma 13
4.3 Öbek sıralaması 16
4.4 Sayma sıralaması 18
4.5 Taban sıralaması (LSD) 21
4.6 Kova sıralaması 24
4.7 Bütüncül karşılaştırma (H5–H7) ve algoritma seçimi 26
5. Çözümlü Örnekler 27
5.1 Öbek özelliği denetimi (Örnek 2) 27
5.2 Öncelik kuyruğuyla ilk-k (Örnek 6) 27
5.3 k sıralı listenin birleştirilmesi (Örnek 7) 28
5.4 Sıralama karşılaştırma deneyi: sayaçlar (Örnek 12) 29
6. Haftanın Deneyi: Sekiz Sıralama, Dört Girdi, Anahtar Aralığı 29
6.1 Ölçüm düzeni 30
6.2 Sonuçlar 30
6.3 Kuramla karşılaştırma 31
6.4 Anahtar aralığına duyarlılık 32
6.5 Öbek kurma ve diller 32
6.6 Yorum, sınırlılıklar ve rapor nasıl yazılır? 32
7. Hata Avcısı 33
8. Dil Penceresi: C · C++ · Python 34
9. Mühendis Gözüyle: Veri Profiline Göre Seçim 36
10. Ara Proje: Mini-OBS 2 v2.6 ve Sıralama Ölçüm Raporu 36
10.1 Kapsam: v2.3–v2.6 36
10.2 v2.6 gereksinimleri ve ipuçları 37

10.3 Kabul ölçütleri ve test senaryoları 37
10.4 Ölçüm raporu 38
10.5 Puanlama 39
11. Alıştırmalar ve Atölyeler 39
11.1 İzleme ve çıktı tahmini (A) 40
11.2 Karmaşıklık (B) 40
11.3 Değişmez yazma (C) 40
11.4 Hata bulma (D) 41
11.5 Parsons (E) 42
11.6 Kod yazma (F) 42
11.7 Diller arası okuma (G) 43
11.8 Atölye 1 · Öbek oyunu (sınıf, ≈ 15 dk) 43
11.9 Atölye 2 · Algoritma seçimi: sekiz senaryo kartı (gruplar, ≈ 20 dk) 43
11.10 Atölye 3 · Ara proje çalışma oturumu ve kod incelemesi (eşli, ≈ 30 dk) 44
12. Kendini Değerlendirme ve Vize Hazırlığı 44
12.1 Bu hafta 44
12.2 Vize hazırlığı (H1–H7) 45
13. İleri Bakış ve Merak Kutusu 46
14. Özet, Kavram Haritası ve Terim Sözlüğü 47
15. Kaynaklar 48
Ek A. Dil Karşılıkları 49
A.1 Örnek 1 · Dizi ↔ ağaç 49
A.2 Örnek 2 · Öbek özelliği denetimi 50
A.3 Örnek 3 · Ekleme ve en büyüğü çıkarma 50
A.4 Örnek 4 · Aşağıdan yukarıya öbek kurma 52
A.5 Örnek 5 · Öbek sıralaması 53
A.6 Örnek 6 · Öncelik kuyruğuyla ilk-k 53
A.7 Örnek 7 · k sıralı listenin birleştirilmesi 55
A.8 Örnek 8 · Sayma sıralaması 56
A.9 Örnek 9 · Kararlı sayma sıralaması 57
A.10 Örnek 10 · Taban sıralaması (LSD) 58
A.11 Örnek 11 · Kova sıralaması 59
A.12 Örnek 12 · Sıralama karşılaştırma deneyi 60
A.13 Örnek 13 · Akan veride ortanca 61

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Karşılaştırma yapmadan sıralamak mümkün mü ve Ω(n log n) sınırı nasıl “aşılır”?

H6'da karşılaştırmaya dayalı her sıralamanın en kötü durumda en az $\lceil \log_2 n! \rceil \approx n \log_2 n$ karşılaştırma yapmak zorunda olduğunu gördük. Oysa bu hafta yazacağımız **sayma sıralaması** (counting sort), 10^6 rastgele sayıyı (anahtarlar 0 … $10^6 - 1$) bu makinede 18,7 ms'de sıralıyor; C kütüphanesinin `qsort`'u aynı işi 142,0 ms'de yapıyor (Bölüm 6). Üstelik sayma sıralaması tek bir anahtar karşılaştırması yapmaz. Alt sınır yanlış mı? Hayır: alt sınır yalnızca elemanlar hakkında “a < b mi?” sorusuyla bilgi edinen algoritmalar içindir. Sayma sıralaması anahtarın **kendisini** dizi indisi olarak kullanır; bunun bir bedeli ve koşulları vardır. Bu haftanın ikinci yarısı o koşulları inceler.

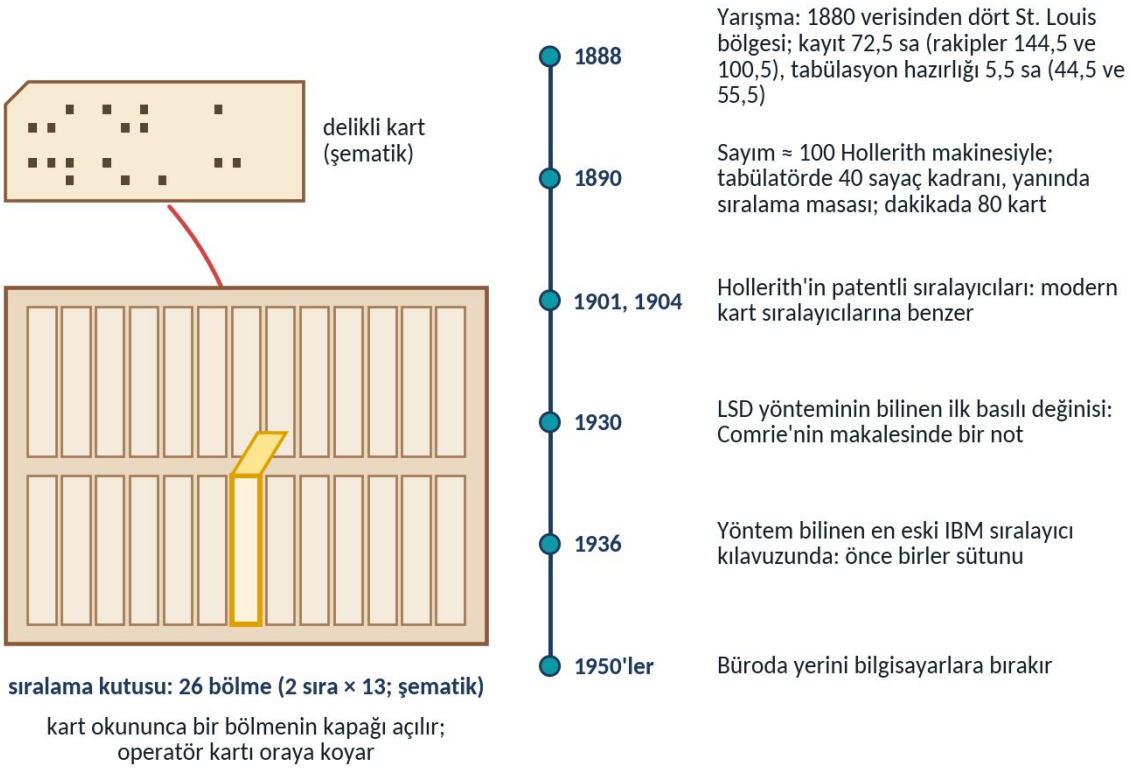
Haftanın ilk yarısı ise bir veri yapısıyla başlar: **ikili öbek** (binary heap). Öbek, tam ikili ağacı hiçbir işaretçi kullanmadan bir dizide tutar ve “en büyüğü ver” sorusunu O(1)'de, ekleme ve çıkarmayı O(log n)'de yanıtlar. Bu yapı üç şeyi birden verir: yerinde ve en kötü durumda O(n log n) bir sıralama (**öbek sıralaması**, heapsort), birçok algoritmanın aracı olan **öncelik kuyruğu** (priority queue) ve “bütün listeyi sıralamadan ilk k'yı bul” gibi problemlerin çözümü. Hafta, H5–H7'nin bütün sıralama algoritmalarını tek tabloda karşılaştırıp **veri profiline göre seçim** yaparak ve ara projeyle biter.

> **GERÇEK DÜNYADAN · 1890: kartları sütun sütun ceplere ayırmak**
>
> Herman Hollerith’in 1890 ABD nüfus sayımı için geliştirdiği delikli kartlı tabülasyon sistemi ve sonraki on yıllarda kartları sütun sütun ceplere ayıran mekanik kart sıralayıcıları: en düşük anlamlı basamaktan başlayan taban sıralamasının (LSD radix sort) fiziksel gerçekleştirimi (Knuth, 1998).
>
> 1888'de ABD Nüfus Sayım Bürosu'nun düzenlediği yarışmada Hollerith'in sistemi 1880 sayımından dört St. Louis bölgesinin verisini 72,5 saatte kaydetti (rakipler 144,5 ve 100,5 saat); tabülasyon hazırlığı 5,5 saat sürdü (rakipler 44,5 ve 55,5 saat). 1890 tabülatörlerinde 40 sayaç kadranı vardı, her tabülatörün yanında bir sıralama masası dururdu ve deneyimli bir memur dakikada 80 kart işleyebiliyordu; teknoloji büroda 1950'lerde bilgisayarlar gelene dek kullanıldı (U.S. Census Bureau, 2024).
>
> Knuth'a (1998) göre 1890 sayımında yaklaşık 100 Hollerith makinesi kullanıldı; ilk aygıtta 26 bölmeli bir sıralama kutusu vardı: kart okununca bir bölmenin kapağı açılır, operatör kartı oraya koyardı. Hollerith'in patenti iki sütunlu sayıların her sütun için ayrı sıralanacağını söylüyor, ama hangi sütunun önce geleceğini belirtmiyordu. **Önce birler sütununa göre dağıtıp kartları sırayla toplamak, sonra onlar sütununa geçmek** hilesini büyük olasılıkla adı bilinmeyen makine operatörleri buldu; yöntem bilinen en eski IBM sıralayıcı kılavuzunda (1936) yer alır, basılı ilk değinisi L. J. Comrie'nin 1930 tarihli bir makalesindeki rastlantısal bir nottur. Bu, Bölüm 4.5'teki LSD taban sıralamasıdır (Şekil 7.1).

1.1 Öğrenme çıktıları

- **ÖÇ 7.1** Tam ikili ağacın dizi gösterimini (ebeveyn ve çocuk indisleri) kullanarak ikili öbek işlemlerini (ekleme, en büyüğü çıkarma, yukarı/aşağı süzme) gerçekler. (Bölüm 3.1, 4.1, Örnek 1–3)
- **ÖÇ 7.2** Aşağıdan yukarıya öbek kurmanın O(n), öbek sıralamasının O(n log n) olduğunu açıklar; öbek sıralamasını gerçekler. (Bölüm 4.2–4.3, Örnek 4–5)
- **ÖÇ 7.3** Öncelik kuyruğunu ilk-k ve çok yollu birleştirme problemlerine uygular. (Bölüm 5.2–5.3, Örnek 6–7)
- **ÖÇ 7.4** Sayma ve taban (LSD) sıralamasını gerçekler; kararlılığın taban sıralamasının doğruluğundaki rolünü açıklar. (Bölüm 4.4–4.5, Örnek 8–10)
- **ÖÇ 7.5** Veri özelliklerine (boyut, anahtar aralığı, neredeyse sıralılık, kararlılık gereksinimi, bellek) göre uygun sıralama algoritmasını gerekçeli olarak seçer. (Bölüm 4.7, 6 ve 10, Örnek 12)

Şekil 7.1
Hollerith'in sıralama kutusu (şematik çizim) ve doğrulanmış olayların zaman çizgisi (U.S. Census Bureau, 2024; Knuth, 1998)



Kaynak: U.S. Census Bureau (2024); Knuth (1998, § 5.5). Çizim şematiktir; ölçü ve yerleşim temsilîdir.

2. Geçen Haftadan ve AP-1'den Bağlantı

H5'te üç temel sıralama algoritmasını ve **kararlılığı** (stability) gördük: kararlı bir sıralama eşit anahtarlı kayıtların girdideki göreli sırasını korur. H6'da böl ve fethetle Θ(n log n)'e indik ve karar ağacıyla bunun karşılaştırmalı sıralamalar için aşılamayacağını gösterdik. Bu hafta üç aracı yeniden kullanıyoruz: H6'nın karar ağacı (alt sınırın kapsamını belirlemek için), H5'in kararlılık kavramı (taban sıralamasının doğruluğu ona dayanır) ve AP-1 H10'daki **frekans dizisi** (sayma sıralamasının ilk yarısı odur).

> **TAHMİN ET · Geri çağırma: üç soru (yanıtlar aşağıda)**
>
> - **S1 (H6).** n = 8 farklı elemanı sıralayan karşılaştırmalı bir algoritma en kötü durumda en az kaç karşılaştırma yapmak zorundadır?
> - **S2 (H5).** (70, Ada) (85, Banu) (70, Cem) kayıtları nota göre **kararlı** bir algoritmayla artan sıralanırsa çıktı nedir? Kararsız bir algoritma hangi çıktıyı da verebilir?
> - **S3 (AP-1 H10).** 0–100 arasındaki notların kaçar kez geçtiğini `sayac[not]++` ile sayan bir frekans dizisi kaç elemanlı olmalıdır?

Yanıtlar. S1: $\lceil \log_2 8! \rceil = \lceil \log_2 40\,320 \rceil = 16$; birleştirmeli sıralamanın en kötü durumu 17, araya eklemeninki 28 karşılaştırmadır (Kavram 4). S2: kararlı: (70, Ada) (70, Cem) (85, Banu); kararsız bir algoritma iki 70'liyi ters sırada da verebilir. Örnek 5'te öbek sıralamasının tam bunu yaptığını göreceğiz. S3: 101 eleman (indis 0 … 100). `sayac[not]` ifadesi değeri **indis** olarak kullanır: sayma sıralamasının bütün fikri bu satırdadır.

3. Kavramsal Anlatım: Dizideki Ağaç, Öncelik ve Alt Sınırın Kapsamı

3.1 Tam ikili ağaç ve dizi gösterimi

Her düzeyi (belki son düzey dışında) tamamen dolu ve son düzeyi soldan sağa doldurulmuş ikili ağaca **tam ikili ağaç** (complete binary tree) denir. Böyle bir ağacın düğümlerini düzey düzey, soldan sağa numaralandırırsak hiçbir işaretçiye gerek kalmaz: düğüm i'nin çocukları ve ebeveyni indis aritmetiğiyle bulunur. C, C++ ve Python dizileri 0 tabanlı olduğu için bu derste formüller şunlardır:

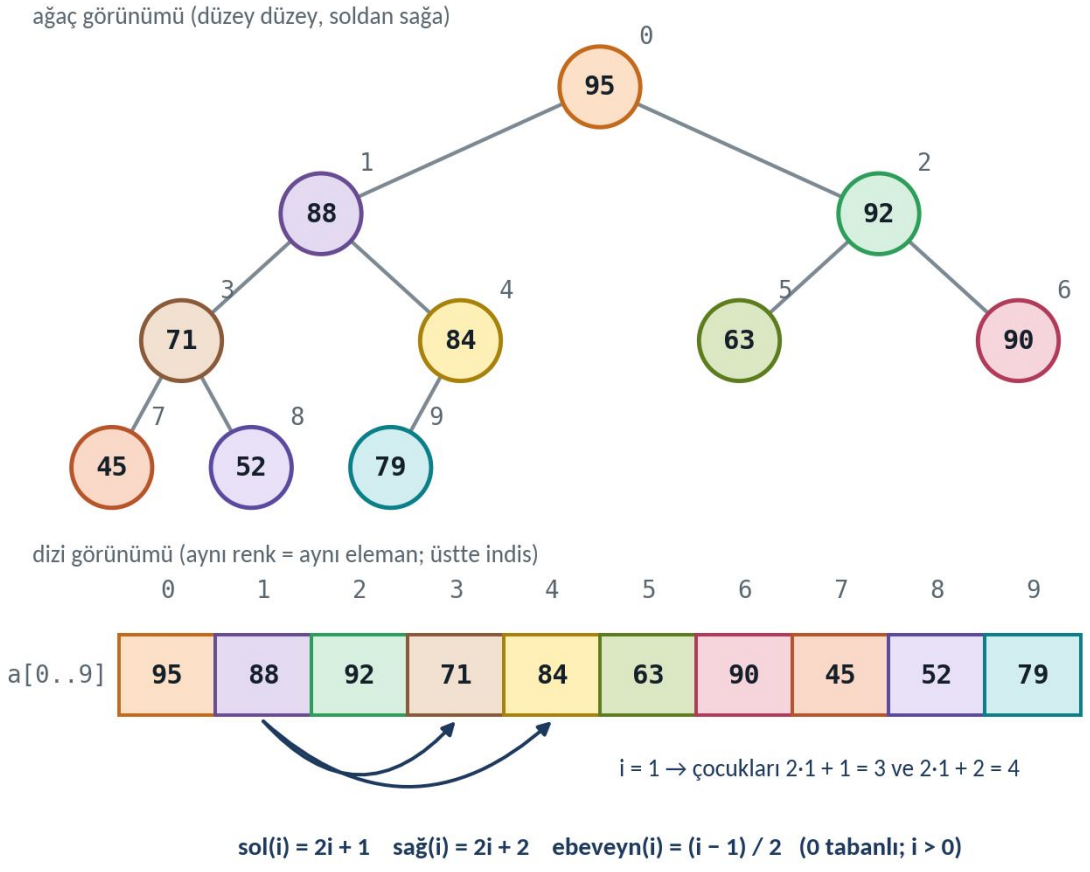
sol(i) = 2i + 1, sağ(i) = 2i + 2, ebeveyn(i) = ⌊(i − 1) / 2⌋ (i > 0); d. düzey: a[$2^d - 1$ … $2^{d+1} - 2$].

Örnek 1 bu üç formülü bir öbek dizisi üzerinde tablo, düzey listesi ve iki ağaç çizimiyle gösterir (Şekil 7.2); aynı eleman dizide ve ağaçta aynı renkle çizilir. n düğümlü tam ikili ağacın yüksekliği ⌊$\log_2 n$⌋'dir: `düzey sayısı (yükseklik + 1): n = 10 -> 4, n = 1000000 -> 20`. Bu, öbek işlemlerinin O(log n) olmasının tek nedenidir.

c/H07_01_dizi_agac.c · satır 16–19

```c
/* 0 tabanlı indis formülleri */
static int ebeveyn(int i) { return (i - 1) / 2; }
static int sol_cocuk(int i) { return 2 * i + 1; }
static int sag_cocuk(int i) { return 2 * i + 2; }
```

Şekil 7.2
Dizi ve ağaç görünümü eşzamanlı (Örnek 1): aynı eleman iki görünümde aynı renk; düğüm yanındaki sayı dizideki indis, kutular üstündeki sayı indis



> KAPUTUN ALTINDA • **İşaretçisiz ağaç ve bit kaydırma**
>
> Öbeğin dizi temsili bağlı bir ağacın üç işaretçisini (sol, sağ, ebeveyn) ortadan kaldırır: düğüm başına bellek yalnızca anahtarın kendisidir ve komşu düzeyler bellekte ardışıktır. İndis aritmetiği bit kaydırmayla da yazılabilir: 2i + 1 = (i << 1) | 1, 2i + 2 = (i + 1) << 1, (i − 1) / 2 = (i − 1) >> 1 (i > 0). Örnek 1 üç formülün bu biçimlerle aynı sonucu verdiğini bir milyon indis için denetler: `i = 0 .. 999999 için tutarsızlık sayısı: 0`. Derleyiciler `2 * i` ve `/ 2`'yi işaretsiz ya da negatif olmadığı bilinen tamsayılarda zaten kaydırmaya çevirir; kaynak kodda okunur biçimi yeğleyin.
>
> **Dikkat:** C'de `(0 - 1) / 2` sıfıra doğru kesildiği için 0, Python'da `(0 - 1) // 2` aşağı yuvarlandığı için −1'dir; kökün "ebeveyni" iki dilde farklı çıkar. Bu yüzden `ebeveyn(i)` yalnızca i > 0 iken çağrılır (Ek A.1).

3.2 Öbek özelliği ve öncelik kuyruğu

Bir dizi, her i > 0 için **a[ebeveyn(i)] ≥ a[i]** koşulunu sağlıyorsa bir **en büyük öbektir** (max-heap); eşitsizlik ters çevrilirse **en küçük öbek** (min-heap) elde edilir. Öbek özelliği yalnızca dikey bir sıradır: her düğüm bütün torunlarından büyük-eşittir, ama kardeşler ve kuzenler arasında hiçbir sıra yoktur. Bu yüzden öbek **sıralı bir dizi değildir** ve kökü (a[0]) dizinin en büyük elemanıdır. Öbek özelliğini denetlemek n − 1 karşılaştırma ister (Örnek 2: `Örnek 1'in öbeği (n = 10): öbek: evet, karşılaştırma = 9`). Kaç dizi öbektir? Örnek 2, 1 … n'nin bütün permütasyonlarını dener: n = 8 için 40 320 permütasyonun yalnızca 210'u öbektir (%0,52). Bu sayılar OEIS A056971 dizisiyle aynıdır (OEIS Foundation Inc., t.y.).

Öncelik kuyruğu (priority queue) her elemanın bir önceliği olduğu ve her zaman en öncelikli elemanın çıkarıldığı bir soyut veri tipidir (ADT). Kuyruk (FIFO) ve yığın (LIFO) gibi bir **arayüzdür**; öbek, onun en yaygın gerçekleştirimidir. Tablo 7.1 işlemleri ve üç gerçekleştirimin maliyetini verir: sırasız dizide ekleme ucuz, çıkarma pahalıdır; sıralı dizide tersi; öbek ikisini dengeler.

Tablo 7.1
Öncelik kuyruğu soyut veri tipi: işlemler ve üç gerçekleştirimin en kötü durum maliyeti

İşlem	Anlamı	Sırasız dizi	Sıralı dizi	İkili öbek
ekle(x)	x'i önceliğiyle kuyruğa koy	O(1)	O(n)	O(log n)
en_buyuk()	en öncelikliyi gör (çıkarma)	O(n)	O(1)	O(1)
en_buyugu_cikar()	en öncelikliyi al ve sil	O(n)	O(1)	O(log n)
kur(a[0..n−1])	n elemandan kuyruk oluştur	O(1)	O(n log n)	Θ(n) (Bölüm 4.2)
bos_mu(), boyut()	eleman var mı, kaç tane?	O(1)	O(1)	O(1)

C'de öncelik kuyruğu elle yazılır: Kavram 7 öbeği `realloc` ile büyüyen bir dizi ve ayrı tutulan boyutla gerçekler (`kapasite = 8, eleman = 8, en büyük = 9; çıkarma sırası: 9 9 8 7 4 3 2 1`). C++'ta `std::priority_queue` (varsayılan **en büyük** öbek), Python'da `heapq` modülü (bir liste üzerinde **en küçük** öbek) vardır (Bölüm 8).

> NOT • **Terim: öbek ve yığın**
>
> "Öbek" (heap) bu derste iki şeyi anlatır: bu haftanın **veri yapısını** ve AP-1'den bildiğimiz `malloc`/`free` ile yönetilen **dinamik bellek bölgesini**. İkisi yalnızca adı paylaşır; `malloc`'un öbeğinin öbek özelliğiyle bir ilgisi yoktur. "Yığın" (stack) ise her zaman son giren ilk çıkar (LIFO) yapısı ya da çağrı yığınıdır (H3, H9). Bağlam belirsizse "öbek veri yapısı" ya da "öbek bellek bölgesi" deriz (Yol Haritası Bölüm 6.8).

3.3 Ω(n log n) alt sınırı ve “aşılması”nın koşulları

H6'da gördüğümüz karar ağacını hatırlayalım (Şekil 7.3, bu alt bölümün sonunda). Karşılaştırmaya dayalı bir sıralama algoritması, elemanlar hakkında yalnızca “a < b mi?” sorularının yanıtlarıyla bilgi edinir. Algoritmanın n elemanlı her girdide sorduğu sorular bir ikili ağaç oluşturur: iç düğümler sorular, dallar yanıtlar, yapraklar çıktı sıralamalarıdır. n farklı elemanın n! düzeninin her biri ayrı bir yaprakta bitmelidir (iki farklı düzen aynı yaprağa düşerse algoritma birini yanlış sıralar). h yükseklikli ikili ağacın en çok 2^h yaprağı olduğundan $2^h \geq n!$, yani **$h \geq \log_2 n!$** olur ve en kötü durumdaki karşılaştırma sayısı yüksekliktir. n = 3 için 6 yaprak ve $3 = \lceil \log_2 6 \rceil$ karşılaştırma; n = 8 için $\lceil \log_2 8! \rceil = 16$.

$\log_2 n! = \log_2 1 + \log_2 2 + \ldots + \log_2 n$ toplamı $n \log_2 n$'den yalnızca doğrusal bir terim kadar küçüktür ($\log_2 n! \approx n \log_2 n - 1{,}44\, n$); oran n = 10'da 0,656, n = 1 000'de 0,856, n = 100 000'de 0,913 olur. Dolayısıyla karşılaştırmalı sıralama Ω(n log n)'dir; birleştirmeli sıralama ve öbek sıralaması bu anlamda **asimptotik olarak en iyidir**.

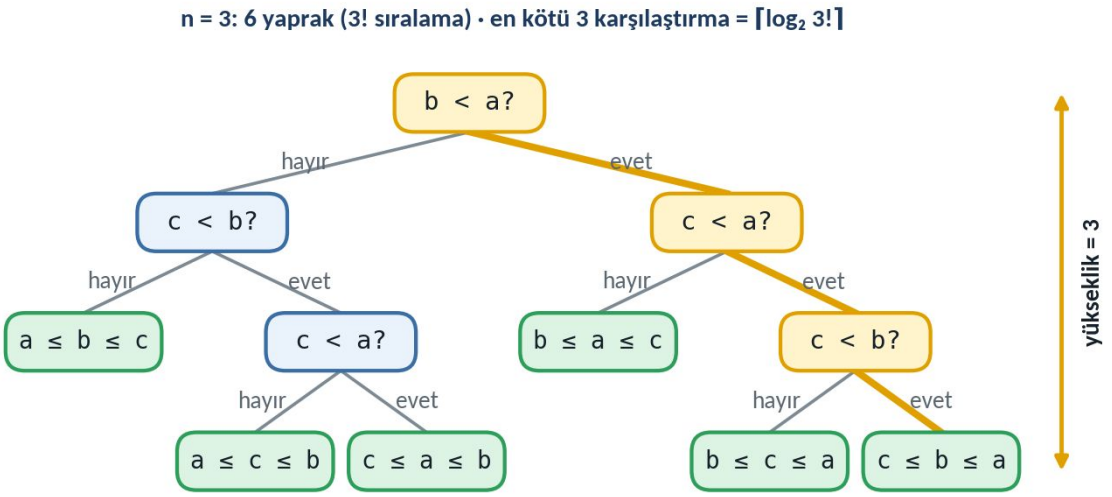
Peki sayma sıralaması nasıl daha hızlı olabiliyor? Karar ağacı kanıtı bir **model** varsayar: algoritmanın elemanlara dokunmasının tek yolu iki elemanı karşılaştırmaktır. Sayma sıralamasındaki `sayac[a[i]]++` satırı bu modelin dışındadır: tek adımda a[i]'nin değerine göre k + 1 yoldan birine dallanır (bir karşılaştırma ise yalnızca iki yola). Taban sıralaması aynı işi basamak basamak yapar; kova sıralaması değeri bir aritmetik işlemle kova numarasına çevirir. Alt sınır “aşılmaz”, kapsamı dışına çıkılır. Bunun koşulları vardır:

- **Anahtar bir tamsayı ve aralığı küçük olmalı** (sayma: $k \lesssim n$). 32 bitlik anahtarlarda sayaç dizisi $2^{32} \times 4$ bayt = 16 GiB olurdu (Bölüm 3.4).
- **Anahtar sabit uzunlukta basamaklara ayrılabilmeli** (taban: d basamak, taban b). Her basamakta kararlı sayma yapılır; maliyet Θ(d(n + b)).
- **Dağılım bilinmeli** (kova: [0, 1) aralığında yaklaşık düzgün). Varsayım bozulursa kova sıralaması $\Theta(n^2)$'ye düşer (Bölüm 4.6).
- **Ω(n) her durumda kalır:** her elemanı en az bir kez okumadan hiçbir algoritma sıralayamaz. Doğrusal zamanlı sıralamalar bu sınıra ulaşır.

3.4 Kaputun Altında: önbellek ve sayaç dizisinin belleği

> KAPUTUN ALTINDA • **Öbek sıralaması neden hızlı sıralamadan yavaş? Sayma neden 32 bitte uygulanamaz?**
>
> **Önbellek.** Aşağı süzme a[i]'den a[2i + 1]'e atlar: düzey indikçe adımlar ikiye katlanır ve büyük dizide ardışık erişimler farklı önbellek satırlarına düşer. Hızlı sıralama ise diziyi iki uçtan ardışık tarar. Bu makinede C -O2, $n = 10^6$ rastgele girdide öbek sıralaması 207,2 ms, hızlı sıralama 101,9 ms: ≈ 2 kat yavaş (iki ölçüm oturumunda 1,75–2,52 kat; işlemci zamanı). n log n modeli 10^6 ölçümünden 10^7 için 2,42 s kestirdi, ölçülen 3,67 s oldu (hata %−34,1): dizi önbelleğe sığmadıkça öbek sıralaması daha da yavaşlar. Karşılaştırma sayısı benzer olduğu hâlde (Örnek 12) aradaki fark bellek erişiminden gelir. Öbek sıralamasının asıl değeri **güvencedir**: en kötü durumda da O(n log n) ve O(1) ek bellek. Bu yüzden introsort (Musser, 1997) hızlı sıralamayla başlar, özyineleme derinliği $2\lfloor \log_2 n \rfloor$'yi aşınca öbek sıralamasına geçer; libstdc++'ın `std::sort`'u böyle çalışır.
>
> **Sayaç dizisinin belleği.** Sayma sıralaması k + 1 sayaç ayırır ve hepsini tarar: $n = 10^6$, $K = 10^8$ anahtar aralığında C'de sayma 530,5 ms, taban sıralaması 22,9 ms sürdü; sayaç dizisi 400 MB'tır ve önbelleğe sığmaz. $K = 2^{32}$'de sayaç dizisi 16 GiB olurdu (deneyde “uygulanamaz”). Taban sıralaması 32 bitlik anahtarı dört bayta böler: her geçişte yalnızca 256 sayaç (1 KiB) kullanır, dört geçişte biter. Python'da liste yuvası 8 baytlık bir başvurudur; aynı sayaç dizisi en az iki kat yer kaplar.

Şekil 7.3
n = 3 için karar ağacı (iç düğüm: soru; yaprak: çıktı düzeni) ve n = 1 … 8 için alt sınırın gerçek algoritmalarla karşılaştırılması (Kavram 4)



n	1	2	3	4	5	6	7	8
n! (yaprak sayısı)	1	2	6	24	120	720	5040	40320
⌈log₂ n!⌉ (alt sınır)	**0**	**1**	**3**	**5**	**7**	**10**	**13**	**16**
araya ekleme · en kötü	0	1	3	6	10	15	21	28
birleştirmeli · en kötü	0	1	3	5	8	11	14	17

$\log_2 n! / (n \log_2 n)$: 0,656 (n = 10) · 0,856 ($n = 10^3$) · 0,913 ($n = 10^5$) → 1'e yaklaşır: $\Omega(n \log n)$

4. Algoritmalar

Altı algoritmayı aynı on bir adımlı standart anlatımla işliyoruz: problem, fikir, elle izleme, sözde kod, doğruluk, analiz (ve kimlik kartı), C gerçekleştirimi, üç dil karşılaştırması, test, deney ve varyantlar. İlk üçü öbeğe (4.1–4.3), son üçü karşılaştırmasız sıralamaya (4.4–4.6) aittir; Bölüm 4.7 H5–H7'nin bütün sıralamalarını tek tabloda karşılaştırır. Öbek örneklerinde aynı on bir değer (45 … 97) kullanılır ve her şekilde aynı değer aynı renktedir.

4.1 Öbeğe ekleme ve en büyüğü çıkarma (yukarı ve aşağı süzme)

1 · Problem. Bir en büyük öbeğe yeni eleman eklemek ve en büyük elemanı (kökü) çıkarmak; her iki işlemden sonra dizi yine öbek olmalıdır. **2 · Fikir.** Öbeğin **biçimi** (tam ikili ağaç) yalnızca dizinin sonunda değişebilir: ekleme yeni elemanı ilk boş yaprağa (a[n]) koyar, çıkarma son yaprağı (a[n − 1]) köke taşır. Biçim böylece korunur; bozulan yalnızca **sıradır** ve tek bir yol boyunca onarılır. Eklenen eleman ebeveyninden büyük oldukça onunla yer değiştirir (**yukarı süzme**, sift-up); köke taşınan eleman iki çocuğunun **büyüğünden** küçük oldukça onunla yer değiştirir (**aşağı süzme**, sift-down).

3 · Elle izleme. Örnek 3 boş öbeğe on değeri ekler; sonuç Örnek 1'in öbeğidir. Ardından 97 eklenir ve köke kadar çıkar: `ekle 97: konum 10 -> 0, takas = 3 (düzey sayısı - 1) [97 95 92 71 88 63 90 45 52 79 84]`. İlk çıkarmada 97 alınır, son yaprak 84 köke taşınır ve iki düzey iner: `çıkar -> 97 (köke 84 taşındı), karşılaştırma = 5, takas = 2` (Şekil 7.4).

TAHMİN ET · Sizce nerede durur?

[90 79 63 45] öbeğine 84 eklenince 84 hangi indiste durur ve kaç takas olur? (Yanıt: 84 önce a[4]'e konur; ebeveyni a[1] = 79'dan büyük olduğu için takaslanır, yeni ebeveyni a[0] = 90'dan küçük olduğu için durur: `ekle 84: konum 4 -> 1, takas = 1 [90 84 63 45 79]`.)

SÖZDE KOD · 4 · Sözde kod: yukarı ve aşağı süzme (ekleme: a[n] ← x; YUKARI-SÜZ(a, n) · çıkarma: a[0] ← a[n − 1]; AŞAĞI-SÜZ(a, n − 1, 0))

```
algoritma YUKARI-SÜZ(a, i)                ▷ a[0..i−1] öbek, a[i] yeni eleman
    iken i > 0 ve a[i] > a[(i − 1) / 2]
        a[i] ↔ a[(i − 1) / 2];  i ← (i − 1) / 2
algoritma AŞAĞI-SÜZ(a, n, i)              ▷ i'nin iki alt ağacı öbek
    iken 2i + 1 < n
        c ← 2i + 1                         ▷ sol çocuk
        eğer c + 1 < n ve a[c + 1] > a[c] ise c ← c + 1
        eğer a[c] ≤ a[i] ise dur           ▷ öbek özelliği sağlandı
        a[i] ↔ a[c];  i ← c
```

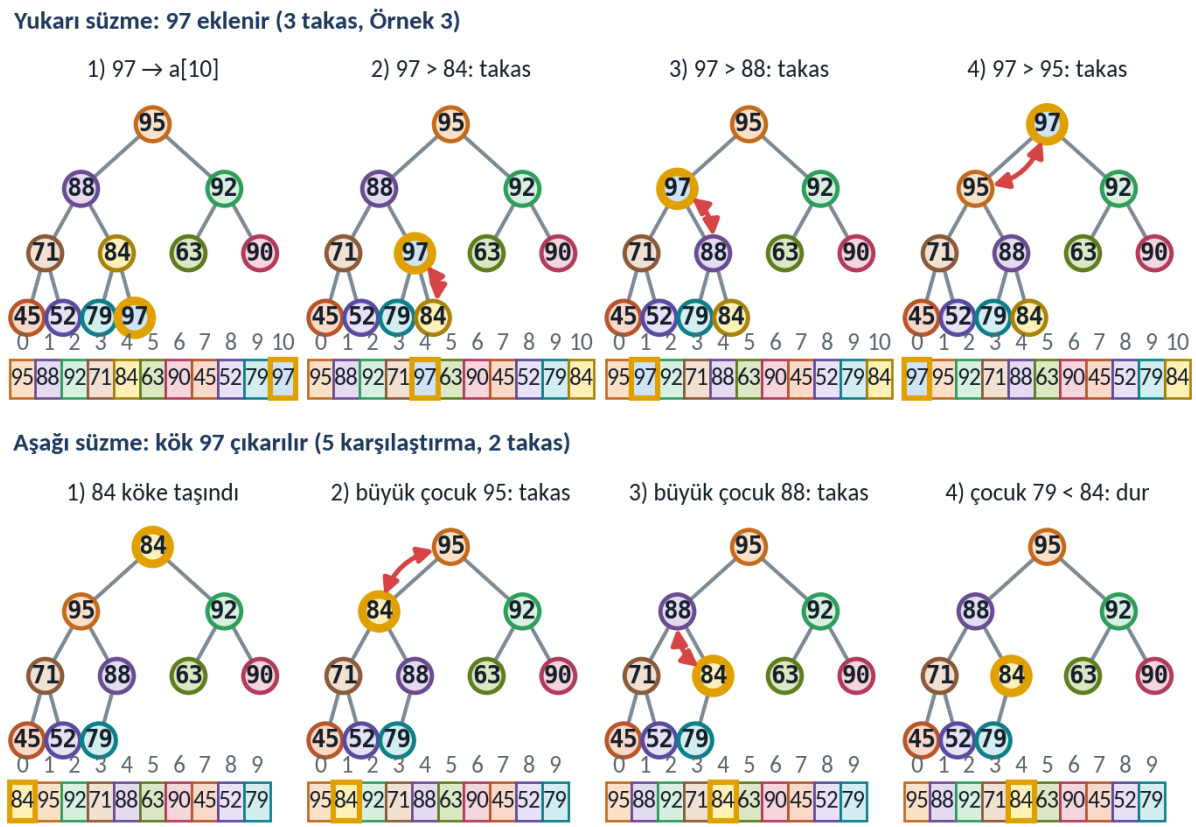
5 · Doğruluk. Yukarı süzmenin değişmezi (Tablo 7.2): Yukarı süzme sırasında öbek özelliği yalnızca i ile ebeveyni arasında bozuk olabilir; diğer tüm ebeveyn–çocuk çiftleri sağlanır. Takas, bozuk çifti düzeltir ve bozukluğu bir düzey yukarı taşır; i = 0'da ya da a[i] ≤ ebeveyn olduğunda bozuk çift kalmaz. Aşağı süzmenin değişmezi (Tablo 7.3): Aşağı süzmede öbek özelliği yalnızca i ile çocukları arasında bozuk olabilir; i'nin iki alt ağacı öbektir. **Büyük** çocuk seçilmelidir: küçük çocuk yukarı çıkarsa büyük kardeşinin ebeveyni olur ve özellik bozulur (Hata Avcısı 2). **6 · Analiz.** Her iki süzme en çok ağacın yüksekliği $\lfloor \log_2 n \rfloor$ kadar düzey dolaşır: O(log n). Aşağı süzme düzey başına 2 karşılaştırma yapar. Ortalamalar ise farklıdır: n = 100 000 rastgele anahtarda ekleme başına 1,285 takas (eleman çoğunlukla yaprağa yakın kalır), artan girdide 14,689 ($\log_2 n$ = 16,610); çıkarma başına 14,007 takas ve 28,315 karşılaştırma, çünkü köke taşınan son yaprak küçüktür ve neredeyse dibe iner.

Tablo 7.2
Öbeğe ekleme kimlik kartı

ALGORİTMA KİMLİK KARTI	**Öbeğe ekleme (yukarı süzme)** *(heap insert (sift-up))*
Strateji	Öbek (öncelik kuyruğu): yeni eleman ilk boş yaprağa konur, ebeveyninden büyük oldukça takaslanır
Girdi önkoşulu	a[0..n−1] en büyük öbek; dizide boş yer (C'de kapasite denetimi)
Zaman (en iyi · ortalama · en kötü)	O(1) · O(1) beklenen (rastgele anahtar) · O(log n)
Ek bellek	O(1)
Özellikler	Rastgele anahtarlarda ekleme başına ortalama 1,259–1,285 takas: yeni eleman çoğunlukla yaprak düzeyine yakın kalır. Artan girdide her eleman köke çıkar: n = 10^5'te 14,69 takas/ekleme
Değişmez / doğruluk	Yukarı süzme sırasında öbek özelliği yalnızca i ile ebeveyni arasında bozuk olabilir; diğer tüm ebeveyn–çocuk çiftleri sağlanır
Kütüphane karşılığı	C: yok (elle) · C++: `std::priority_queue::push`, `std::push_heap` · Python: `heapq.heappush` (en küçük öbek)
Ne zaman kullanılır	Öncelik kuyruğu: olay benzetimi, görev zamanlama, akan veride ilk-k ve ortanca (Örnek 6, 13); H12'de Dijkstra, Prim ve Huffman

Ne zaman kullanılmaz	Elemanların tümü baştan biliniyorsa tek tek eklemek yerine aşağıdan yukarıya kurma (Θ(n)); rastgele bir değeri aramak gerekiyorsa (öbekte arama O(n))

Şekil 7.4
Yukarı süzme (üstte: 97 eklenir, üç takasla köke çıkar) ve aşağı süzme (altta: kök 97 çıkarılır, 84 iki düzey iner), Örnek 3; her karede ağaç ve dizi birlikte, etkin eleman sarı kenarlı, takas kırmızı okla



Tablo 7.3
En büyüğü çıkarma kimlik kartı

ALGORİTMA KİMLİK KARTI **En büyüğü çıkarma (aşağı süzme)** *(extract-max (sift-down))*	
Strateji	Öbek (öncelik kuyruğu): kök alınır, son yaprak köke taşınır ve büyük çocuğuyla takasla aşağı süzülür
Girdi önkoşulu	a[0..n−1] en büyük öbek, n ≥ 1
Zaman (en iyi · ortalama · en kötü)	O(1) · Θ(log n) · Θ(log n)
Ek bellek	O(1)
Özellikler	Her düzeyde en çok 2 karşılaştırma ve 1 takas. Örnek 3: n = 10^5 rastgele anahtar tümüyle çıkarılırken çıkarma başına 14,0 takas ve 28,3 karşılaştırma: taşınan son yaprak küçük olduğundan neredeyse dibe iner
Değişmez / doğruluk	Aşağı süzmede öbek özelliği yalnızca i ile çocukları arasında bozuk olabilir; i'nin iki alt ağacı öbektir
Kütüphane karşılığı	C: yok (elle) · C++: `std::priority_queue::top` + `pop`, `std::pop_heap` · Python: `heapq.heappop` (en küçüğü çıkarır)

Ne zaman kullanılır	En öncelikli işi tekrar tekrar almak; öbek sıralamasının ikinci evresi; k yollu birleştirme (Örnek 7)
Ne zaman kullanılmaz	Hem en küçük hem en büyük sık gerekiyorsa tek öbek yetmez (iki öbek: Örnek 13; ya da dengeli arama ağacı, Veri Yapıları dersi)

7 · C gerçekleştirimi. (Örnek 3; sayaçlar karşılaştırma ve takası ayrı sayar, öbek sabit kapasiteli bir dizi ve ayrı tutulan boyuttur):

C `c/H07_03_ekle_cikar.c · satır 37–62`

```c
/* a[i] ebeveyninden büyük oldukça yukarı taşınır; son konumu döndürür */
static int yukari_suz(int *a, int i)
{
    while (i > 0 && buyuk_mu(a[i], a[ebeveyn(i)])) {
        takas(&a[i], &a[ebeveyn(i)]);
        i = ebeveyn(i);
    }
    return i;
}

/* a[i], büyük çocuğundan küçük oldukça aşağı taşınır (a[0..n-1]) */
static int asagi_suz(int *a, int n, int i)
{
    while (2 * i + 1 < n) {
        int c = 2 * i + 1;  /* sol çocuk */
        if (c + 1 < n && buyuk_mu(a[c + 1], a[c])) {
            c++;  /* sağ çocuk daha büyük */
        }
        if (!buyuk_mu(a[c], a[i])) {
            break;  /* öbek özelliği sağlandı */
        }
        takas(&a[i], &a[c]);
        i = c;
    }
    return i;
}
```

Ekleme `a[*n] = x; (*n)++;` ve `yukari_suz(a, *n - 1)`, çıkarma `a[0] = a[*n]` (boyut azaltıldıktan sonra) ve `asagi_suz(a, *n, 0)`'dır (65–80. satırlar). **8 · Üç dil karşılaştırması.** Süzme fonksiyonları üç dilde satır satır aynıdır (Ek A.3); farklılaşan, öbeğin boyutunu kimin tuttuğudur: C'de sabit kapasiteli dizi ve ayrı boyut, C++'ta `std::vector`'ün `back`/`pop_back`'i, Python'da listenin `a[-1]`/`pop`'u:

C++ `H07_03_ekle_cikar.cpp · satır 71–78`

```cpp
static int en_buyugu_cikar(std::vector<int>& a)
{
    int en_buyuk = a[0];
    a[0] = a.back();
    a.pop_back();
    asagi_suz(a, static_cast<int>(a.size()), 0);
    return en_buyuk;
}
```

Python **H07_03_ekle_cikar.py · satır 60–66**

```python
def en_buyugu_cikar(a):
    """Kök çıkarılır; son eleman köke taşınıp aşağı süzülür (a boş olmamalı)"""
    en_buyuk = a[0]
    a[0] = a[-1]
    a.pop()
    asagi_suz(a, len(a), 0)
    return en_buyuk
```

9 · Test. On eklemeden sonra `Örnek 1'in öbeğiyle aynı mı? evet`; kalan elemanlar sırayla çıkarılınca azalan dizi elde edilir (`90 88 84 79 71 63 52 45`): "sırayla çıkarılanlar azalan mı?" öbek işlemlerinin kâhinidir. C++ kütüphanesi aynı işlemleri `std::push_heap`/`std::pop_heap` ile yapar (Kavram 2: `push_heap(97) : [97 95 88 90 92 84 63 45 71 52 79] öbek mi? evet`). **10 · Deney.** Ortalama takas sayıları n'den neredeyse bağımsızdır (n = 10^3 … 10^5: 1,259, 1,268, 1,285); artan girdide $\log_2 n$ ile büyür. **11 · Varyantlar.** En küçük öbek (karşılaştırmayı çevirmek); d-li öbek (Alıştırma 7.12); bir anahtarı artırıp yukarı süzmek (en küçük öbekte anahtar azaltma; H12'de Dijkstra bunun yerine tembel silmeyi kullanır); iki öbekle ortanca (Merak Kutusu).

4.2 Aşağıdan yukarıya öbek kurma

1 · Problem. Sırasız bir diziyi yerinde öbeğe çevirmek. **2 · Fikir.** Elemanları tek tek eklemek (yukarı süzmek) O(n log n) sürer. Oysa yapraklar (i ≥ ⌊n/2⌋) zaten tek elemanlı öbektir; son iç düğümden (⌊n/2⌋ − 1) köke doğru her düğümü aşağı süzmek yeter: bir düğüm süzüldüğünde iki alt ağacı zaten öbektir. Düğümlerin yarısı hiç süzülmez, dörtte biri en çok bir düzey iner.

3 · Elle izleme. Artan girdi zor bir durumdur, çünkü her süzülen eleman alt ağacının en küçüğüdür ve bir yaprağa kadar iner (Şekil 7.5): i = 4, 3, 2 birer, i = 1 iki, i = 0 üç takas yapar; `toplam: karşılaştırma = 14, takas = 8, öbek mi? evet`, `üst sınır Σ yükseklik = 8 (n - popcount(n) = 8)`.

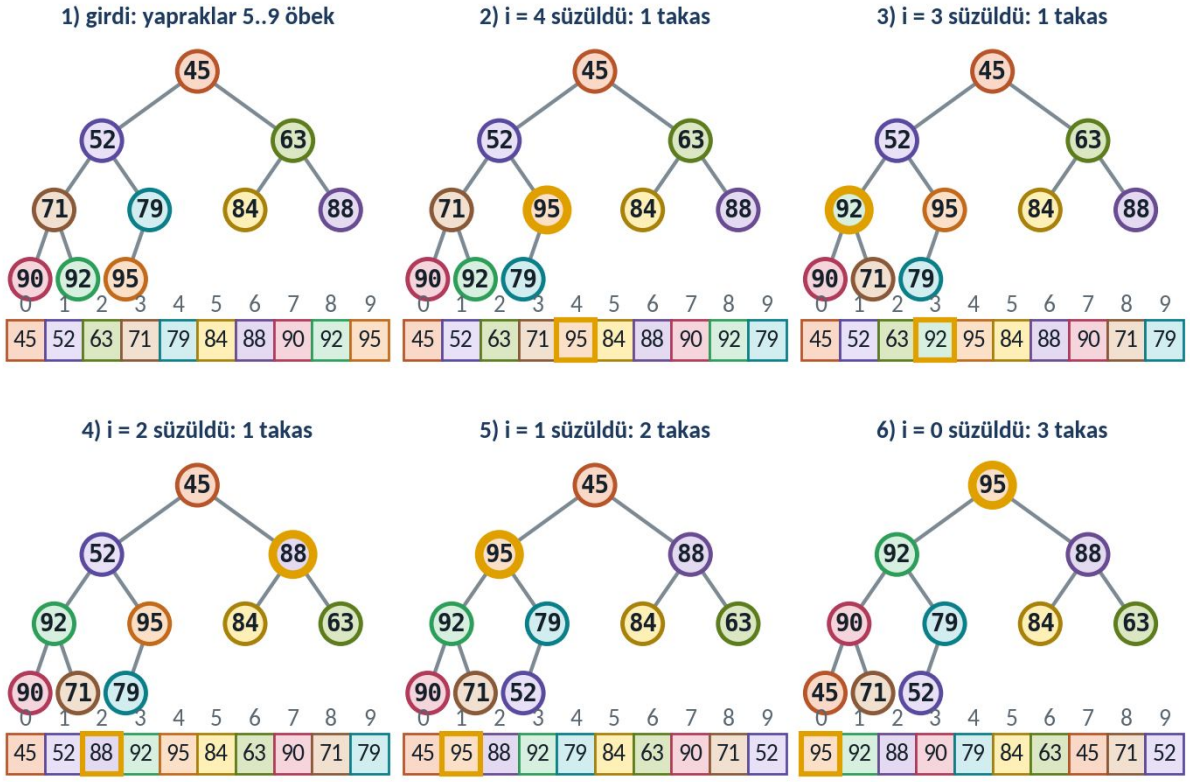
> **TAHMİN ET · Sizce kaç takas?**
>
> i = 1'de [45 52 88 92 95 84 63 90 71 79] dizisinde 52 süzülürken hangi elemanlarla yer değiştirir? (Yanıt: çocukları a[3] = 92 ve a[4] = 95; büyüğü 95 ile takaslanır, sonra a[4]'ün tek çocuğu a[9] = 79 ile: iki takas, [45 95 88 92 79 84 63 90 71 52].)

SÖZDE KOD · 4 · Sözde kod: aşağıdan yukarıya öbek kurma

```
algoritma ÖBEK-KUR(a, n)
    i ← ⌊n / 2⌋ − 1'den 0'a kadar her i için   ▷ yapraklar atlanır
        AŞAĞI-SÜZ(a, n, i)                     ▷ i'nin alt ağaçları zaten öbek
```

5 · Doğruluk. Değişmez (Tablo 7.4): i'den sonraki her düğüm (i + 1, …, n − 1) bir öbeğin köküdür; i süzülünce i de öyle olur. Döngü başında i > ⌊n/2⌋ − 1 olan düğümler yapraktır, değişmez doğrudur; i süzüldüğünde aşağı süzmenin ön koşulu (iki alt ağaç öbek) sağlanmıştır. i = −1'de kök de bir öbeğin köküdür. **i = 0'dan başlamak yanlıştır:** kök süzülürken alt ağaçlar henüz öbek değildir (Hata Avcısı 3). **6 · Analiz.** Yüksekliği h olan düğüm en çok h takas yapar; toplam iş Σ yükseklik ile sınırlıdır ve bu toplam tam olarak n − popcount(n)'dir (popcount: n'nin ikilik yazımındaki 1'lerin sayısı), yani **n'den küçüktür**. Sezgiyi Şekil 7.6 verir: 1023 düğümlü mükemmel ağaçta düzeyler aşağı indikçe düğüm sayısı ikiye katlanır ama iş (yükseklik) birer azalır; üst sınır toplamı 1 013, artan girdide gerçek takas 1 013, rastgele girdide 767. Tek tek eklemeyle aynı artan girdi 19 takas ister; n = 10^5'te eleman başına 14,689 takas ($\log_2 n$ = 16,610): Θ(n log n).

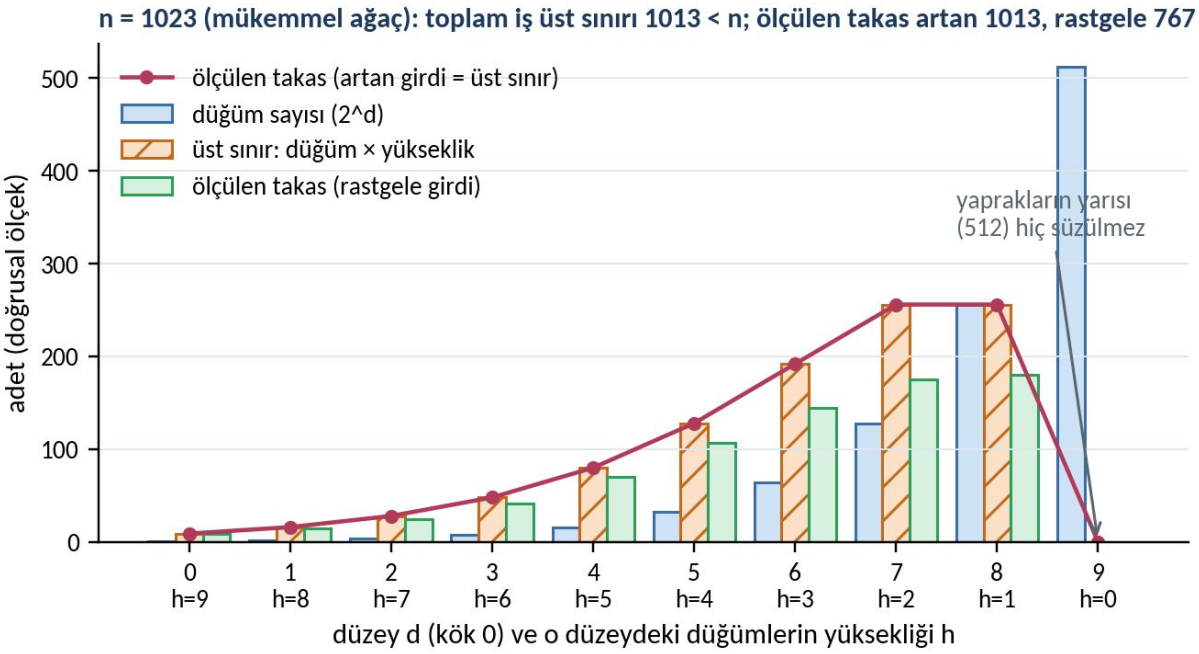
Şekil 7.5
Aşağıdan yukarıya öbek kurma, artan girdi (Örnek 4): yapraklar 5 … 9 baştan öbek; i = 4, 3, 2, 1, 0 sırayla süzülür, toplam 8 takas



Tablo 7.4
Aşağıdan yukarıya öbek kurma kimlik kartı

ALGORİTMA KİMLİK KARTI	**Aşağıdan yukarıya öbek kurma** *(bottom-up heap construction (heapify))*
Strateji	Öbek: yapraklar tek elemanlı öbektir; son iç düğümden (n/2 − 1) köke her düğüm aşağı süzülür
Girdi önkoşulu	Yok — herhangi bir dizi
Zaman (en iyi · ortalama · en kötü)	Θ(n) · Θ(n) · Θ(n)
Ek bellek	O(1)
Özellikler	Takas sayısı ≤ Σ yükseklik = n − popcount(n) < n. Düğümlerin yarısı yapraktır ve hiç süzülmez. Tek tek eklemeyle kurmak artan girdide Θ(n log n): n = 10^5'te 14,69 takas/eleman
Değişmez / doğruluk	i'den sonraki her düğüm (i + 1, …, n − 1) bir öbeğin köküdür; i süzülünce i de öyle olur
Kütüphane karşılığı	C: yok (elle) · C++: `std::make_heap` (en çok 3n karşılaştırma) · Python: `heapq.heapify` (en küçük öbek, doğrusal)
Ne zaman kullanılır	Öbek sıralamasının ilk evresi; tüm elemanlar baştan biliniyorsa öncelik kuyruğunu başlatmak
Ne zaman kullanılmaz	Elemanlar akış hâlinde tek tek geliyorsa (ekleme kullanılır); yalnızca en büyük bir kez gerekiyorsa (doğrusal tarama yeter)

Şekil 7.6
Öbek kurmada düzeylere göre iş (n = 1023, Örnek 4): düğüm sayısı × yükseklik üst sınırı ve ölçülen takaslar; yaprak düzeyi hiç iş yapmaz, toplam < n



7 · C gerçekleştirimi. (Örnek 4; karşılaştırma için tek tek ekleyerek kurma da aynı dosyadadır):

C `c/H07_04_obek_kur.c · satır 61–74`

```c
static void obek_kur(int *a, int n)
{
    for (int i = n / 2 - 1; i >= 0; i--) {
        asagi_suz(a, n, i);
    }
}

/* Yukarıdan aşağıya: a[0..i-1] öbeğine a[i]'yi ekler gibi yukarı süz */
static void obek_kur_ekleyerek(int *a, int n)
{
    for (int i = 1; i < n; i++) {
        yukari_suz(a, i);
    }
}
```

8 · Üç dil karşılaştırması. C++ sürümü C'ninkiyle aynıdır (yalnızca `n` vektörün boyundan alınır; Ek A.4). Geriye sayan döngü Python'da `range` ile yazılır; bitiş değeri dışlayıcı olduğu için −1 verilir:

Python `H07_04_obek_kur.py · satır 52–55`

```python
def obek_kur(a):
    n = len(a)
    for i in range(n // 2 - 1, -1, -1):
        asagi_suz(a, n, i)
```

9 · Test. Program her n için takasın Σ yükseklik = n − popcount(n) sınırını aşmadığını ve sonucun öbek olduğunu denetler (n = 100 000: rastgele 74 588, artan 99 990 takas, sınır 99 994; “evet”, “evet”).

C++'ın `std::make_heap`'i artan girdiden aynı diziyi üretir (Kavram 2: [95 92 88 90 79 84 63 45 71 52]); Python'da `heapq.heapify` en küçük öbek kurar. **10 · Deney.** n = 10^7'ye kadar takas/n hiç 1'i aşmaz (Bölüm 6.5). **11 · Varyantlar.** Öbeği ve öbek sıralamasını Williams (1964) "Algorithm 232: Heapsort" adıyla yayımladı; aşağıdan yukarıya kurma ise aynı yıl Floyd'un (1964) "Treesort 3" adlı ALGOL yordamından gelir (Cormen vd., 2022). C++ standardı `make_heap` için en çok 3n karşılaştırma şart koşar (ISO/IEC, 2017).

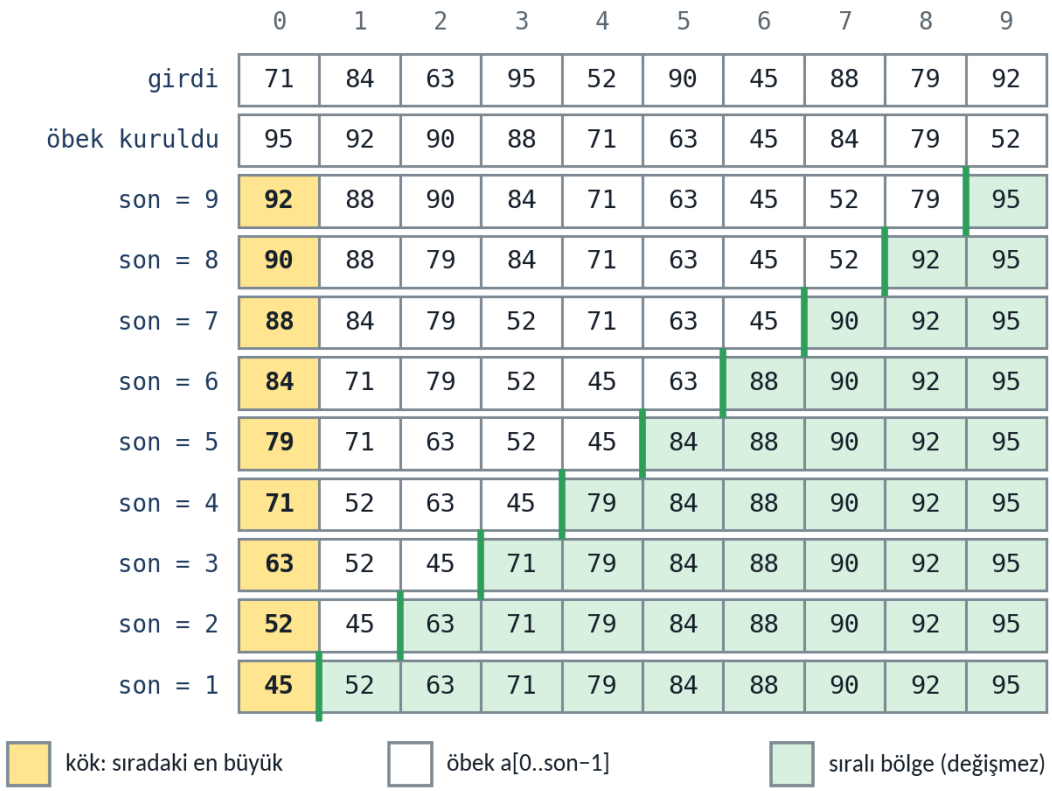
4.3 Öbek sıralaması

1 · Problem. n elemanı yerinde, en kötü durumda O(n log n) sürede sıralamak. **2 · Fikir.** Öbeği kur; kök en büyüktür, onu son konumla takasla. Dizinin sonunda **sıralı bölge** bir eleman büyür, öbek bir küçülür; köke gelen eleman aşağı süzülerek öbek onarılır. Bu, seçmeli sıralamanın (H5) "en büyüğü bul, sona koy" fikridir; fark, en büyüğü O(n) tarama yerine O(log n) süzmeyle bulmaktır.

3 · Elle izleme. Örnek 5'in on elemanlı girdisi `girdi : [71 84 63 95 52 90 45 88 79 92]` önce `öbek kuruldu: [95 92 90 88 71 63 45 84 79 52]` olur; sonraki her satırda kök sıralı bölgenin başına geçer (Şekil 7.7). Toplam: `karşılaştırma = 40, takas = 28 (öbek kurma dahil)`.

> TAHMİN ET · **Sizce sırada ne var?**
>
> `son = 6 : [84 71 79 52 45 63 | 88 90 92 95]` satırından sonra kök hangi elemanla takaslanır ve bir sonraki satır ne olur? (Yanıt: kök 84 son öbek konumu a[5] = 63 ile takaslanır; 63 kökten süzülür: `son = 5 : [79 71 63 52 45 | 84 88 90 92 95]`.)

Şekil 7.7
Öbek sıralaması geçişleri (Örnek 5): sarı kök sıradaki en büyük; yeşil sıralı bölge her adımda bir büyür (değişmez şeridi)



SÖZDE KOD · 4 · Sözde kod: öbek sıralaması

```
algoritma ÖBEK-SIRALAMASI(a, n)
    ÖBEK-KUR(a, n)
    son ← n − 1'den 1'e kadar her son için
        a[0] ↔ a[son]                    ▷ en büyük yerine
        AŞAĞI-SÜZ(a, son, 0)             ▷ a[0..son−1] öbeğini onar
```

5 · Doğruluk. Değişmez (Tablo 7.5): a[son+1..n−1] sıralıdır ve a[0..son] öbeğindeki her elemandan büyük-eşittir; a[0..son] en büyük öbektir. Takas en büyüğü sıralı bölgenin başına koyar; süzme öbeği onarır. son = 0'da sıralı bölge bütün dizidir. **Kararlılık:** takas uzak konumlar arasında yapıldığı için eşit anahtarlıların sırası bozulur; Örnek 5'in ikinci bölümünde `70'ler girdide Ada, Cem, Filiz sırasındaydı: öbek sıralaması kararsızdır`. **6 · Analiz.** Kurma Θ(n), n − 1 süzmenin her biri O(log n): Θ(n log n), girdiden bağımsız. Ek bellek O(1). Sayaçlar uyarlanabilir olmadığını gösterir: n = 10^4'te rastgele girdide 235 458 karşılaştırma (1,772 n $\log_2$ n), **sıralı** girdide 244 460 (1,840 n $\log_2$ n): sıralı girdi kurma aşamasında en çok takası ister.

Tablo 7.5
Öbek sıralaması kimlik kartı

ALGORİTMA KİMLİK KARTI **Öbek sıralaması** *(heapsort)*	
Strateji	Seçme tabanlı (öbekle): öbek kur, kökü (en büyük) sona al, kalan öbeği aşağı süz
Girdi önkoşulu	Yok — karşılaştırılabilir anahtarlar
Zaman (en iyi · ortalama · en kötü)	Θ(n log n) · Θ(n log n) · Θ(n log n)
Ek bellek	O(1)
Özellikler	Kararsız · yerinde · uyarlanabilir değil; bu makinede C -O2, n = 10^6 rastgele girdide 207,2 ms, hızlı sıralama 101,9 ms: ≈ 2 kat yavaş
Değişmez / doğruluk	a[son+1..n−1] sıralıdır ve a[0..son] öbeğindeki her elemandan büyük-eşittir; a[0..son] en büyük öbektir
Kütüphane karşılığı	C: yok (`qsort` algoritmayı belirtmez) · C++: `std::make_heap` + `std::sort_heap`; `std::partial_sort` · Python: `heapq` ile (heapify + n kez heappop; yerinde değil)
Ne zaman kullanılır	En kötü durum O(n log n) ve O(1) ek bellek birlikte gerektiğinde (gömülü sistemler, gerçek zamanlı sınırlar); introsort'un yedek algoritması
Ne zaman kullanılmaz	Kararlılık gerekiyorsa (birleştirmeli); pratik hız önemliyse (hızlı sıralama ya da kütüphane çoğu girdide daha hızlıdır); neredeyse sıralı veride (araya ekleme)

7 · C gerçekleştirimi. (Örnek 5; `asagi_suz` Örnek 3'tekiyle aynıdır):

C c/H07_05_obek_siralamasi.c · satır 59–66

```c
static void obek_siralamasi(int *a, int n)
{
    obek_kur(a, n);
    for (int son = n - 1; son > 0; son--) {
        takas(&a[0], &a[son]);  /* en büyük yerine */
        asagi_suz(a, son, 0);  /* kalan öbeği onar */
    }
}
```

8 · Üç dil karşılaştırması. Fonksiyon üç dilde aynı yapıdadır (C++'ta `std::vector<int>&` parametresi ve `a.size()`); Python'da geriye sayan döngü `range(n - 1, 0, -1)`, takas çoklu atamadır:

Python `H07_05_obek_siralamasi.py · satır 46–51`

```python
def obek_siralamasi(a):
    n = len(a)
    obek_kur(a, n)
    for son in range(n - 1, 0, -1):
        takas(a, 0, son)  # en büyük yerine
        asagi_suz(a, son, 0)  # kalan öbeği onar
```

9 · Test. Her ölçümde sonuç kâhinle (C'de `qsort`) karşılaştırılır; dört girdi türünde "evet". Kararlılık ayrıca sınanır (Örnek 5, 2. bölüm). **10 · Deney.** Bu makinede C -O2, n = 10^6 rastgele girdide 207,2 ms; hızlı sıralama 101,9 ms (Bölüm 3.4 ve 6). **11 · Varyantlar.** Introsort'ta yedek algoritma (Musser, 1997); `std::partial_sort` ve ilk-k (Bölüm 5.2); C++'ta `std::make_heap` + `std::sort_heap` (Kavram 2: `sort_heap : [45 52 63 71 79 84 88 90 92 95] öbek mi? hayır`).

4.4 Sayma sıralaması

1 · Problem. Anahtarları 0 … k aralığında tamsayı olan n kaydı sıralamak. **2 · Fikir.** Karşılaştırma yerine say: `sayac[a[i]]++` her değerin kaç kez geçtiğini bulur; sonra değerler 0'dan k'ya sırayla, sayacı kadar yazılır (Örnek 8). Yalın sayılarda bu yeter. **Kayıtlar** sıralanıyorsa (not ile birlikte öğrenci numarası) her kaydın gideceği konum gerekir: sayacın **önek toplamı** önek[v], anahtarı ≤ v olan kayıt sayısıdır, yani v bloğunun bitişidir. Girdi **sağdan sola** dolaşılır ve her kayıt kendi bloğunun sonundaki boş yere yazılır (Örnek 9, Şekil 7.8).

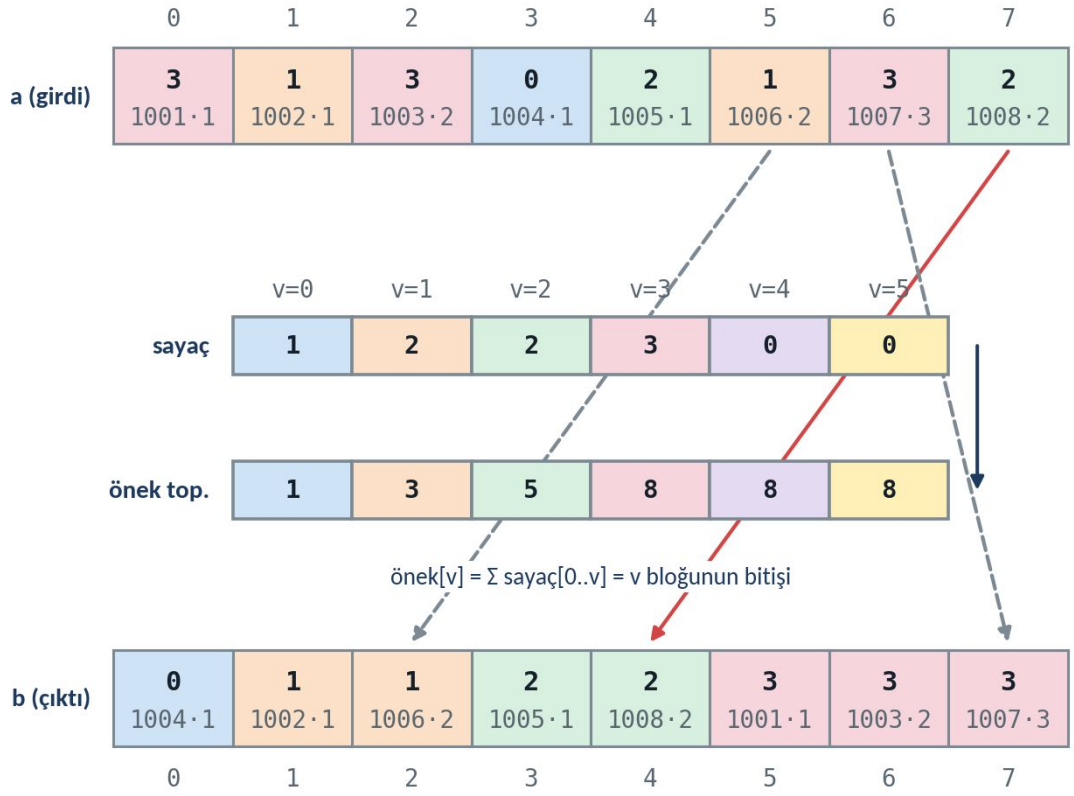
ÇIKTI

```
$ ./H07_08_sayma_siralamasi   (1. bölüm)
  girdi : 85 70 100 45 70 90 85 0 55 70 100 85 62 45 90
  sıfır olmayan sayaçlar: [0]=1 [45]=2 [55]=1 [62]=1 [70]=3 [85]=3 [90]=2 [100]=2
  çıktı : 0 45 45 55 62 70 70 70 85 85 85 90 90 100 100
  adım = 131 = 2n + k + 1 = 131; anahtar karşılaştırması: 0
```

3 · Elle izleme. Örnek 9'da sekiz kayıt (numara, not) ve k = 5: `sayaç : 1 2 2 3 0 0`, `önek top.: 1 3 5 8 8 8`. Yerleştirme a[7]'den başlar: `a[7] = (1008, 2) -> b[4]`, `a[6] = (1007, 3) -> b[7]` (Şekil 7.8).

> TAHMİN ET · **Sizce nereye yazılır?**
>
> Sağdan sola yerleştirmede a[5] = (1006, 1) sırası geldiğinde önek[1] kaçtır, kayıt b'nin hangi konumuna yazılır? (Yanıt: daha önce işlenen a[7] ve a[6]'nın anahtarı 2 ve 3'tür, önek[1] = 3 değişmemiştir; 3 − 1 = 2: `a[5] = (1006, 1) -> b[2]`. Aynı anahtarlı (1002, 1) daha solda olduğu için daha sonra, b[1]'e yazılır.)

Şekil 7.8
Kararlı sayma (Örnek 9): girdi → sayaç → önek toplam → sağdan sola yerleştirme; renk anahtarı, alt yazı kaydın numarası ve eşitler arasındaki girdi sırası (3·1, 3·2, 3·3)



sağdan sola: a[i] → b[--önek[anahtar]]
eşit anahtarlılar (aynı renk) girdi sırasını korur: 3·1, 3·2, 3·3

SÖZDE KOD · 4 · Sözde kod: önek toplamla kararlı sayma sıralaması

```
algoritma KARARLI-SAYMA(a, b, n, k)        ▷ anahtar(a[i]) ∈ 0..k
    önek[0..k] ← 0
    her i için önek[anahtar(a[i])] ← önek[anahtar(a[i])] + 1     ▷ sıklık
    v ← 1'den k'ya kadar her v için önek[v] ← önek[v] + önek[v − 1]
    i ← n − 1'den 0'a kadar her i için     ▷ sağdan sola: kararlı
        v ← anahtar(a[i]);  önek[v] ← önek[v] − 1;  b[önek[v]] ← a[i]
```

5 · Doğruluk. Değişmez (Tablo 7.6): Yerleştirme döngüsünde önek[v], v anahtarlı ve henüz yerleştirilmemiş kayıtların son hedef konumunun bir fazlasıdır. Önek toplamı blokları doğru yerlere koyar (v bloğu önek[v − 1] … önek[v] − 1); bir blok içinde kayıtlar sağdan sola gelip sağdan sola doldurulduğu için girdi sıraları korunur: **kararlıdır**. Aynı önek dizisiyle **soldan sağa** dolaşmak eşitleri ters çevirir: `yol 3 yanlış yön : sıralı mı? evet, kararlı mı? HAYIR` (Örnek 9, yol 3). **6 · Analiz.** Sıklık n, önek k, yerleştirme n adım: **Θ(n + k)** zaman, O(n + k) bellek, sıfır karşılaştırma. Örnek 8 adımları sayar: n = 1 000, k = 100 için 2 101 = 2n + k + 1; k = 1 000 000'da 1 002 001 adım ve 4 000 004 baytlık sayaç dizisi: k ≫ n olunca maliyeti k belirler.

Tablo 7.6
Sayma sıralaması kimlik kartı

ALGORİTMA KİMLİK KARTI	**Sayma sıralaması** *(counting sort)*

Strateji	Karşılaştırmasız: değeri indis olarak kullanma (sıklık dizisi + önek toplam)
Girdi önkoşulu	Anahtarlar küçük bir tamsayı aralığında: 0..k (aralık dışı anahtar: Hata Avcısı 4)
Zaman (en iyi · ortalama · en kötü)	$\Theta(n + k)$ · $\Theta(n + k)$ · $\Theta(n + k)$
Ek bellek	$O(n + k)$
Özellikler	Kararlı · anahtar karşılaştırması yok · $k \gg n$ ise k baskındır: n = 1000, k = 10^6 için 1 002 001 adım; k = 2^{32} için sayaç dizisi 16 GiB olurdu
Değişmez / doğruluk	Yerleştirme döngüsünde önek[v], v anahtarlı ve henüz yerleştirilmemiş kayıtların son hedef konumunun bir fazlasıdır
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (`collections.Counter` yalnızca sayar)
Ne zaman kullanılır	Notlar (0–100), yaşlar (0–120), bayt değerleri gibi dar aralıklar; not dağılımı (histogram); taban sıralamasının her geçişi
Ne zaman kullanılmaz	Anahtar aralığı geniş (32/64 bitlik tamsayı), anahtarlar tamsayı değil (ondalık, dizgi) ya da bellek kısıtlıysa

7 · C gerçekleştirimi. (Örnek 9; `izle` bayrağı izleme satırlarını yazar):

C `c/H07_09_kararli_sayma.c · satır 32–59`

```c
static void kararli_sayma(const Kayit *a, Kayit *b, int n, int k, int izle)
{
    int *onek = calloc((size_t)k + 1, sizeof *onek);
    if (onek == NULL) {
        exit(1);
    }
    for (int i = 0; i < n; i++) {
        onek[a[i].not_]++;                  /* sıklık */
    }
    if (izle) {
        dizi_yaz("sayaç    ", onek, k + 1);
    }
    for (int v = 1; v <= k; v++) {
        onek[v] += onek[v - 1];             /* önek toplam */
    }
    if (izle) {
        dizi_yaz("önek top.", onek, k + 1);
    }
    for (int i = n - 1; i >= 0; i--) {      /* sağdan sola: kararlı */
        int v = a[i].not_;
        onek[v]--;
        b[onek[v]] = a[i];
        if (izle) {
            printf("  a[%d] = (%d, %d) -> b[%d]\n", i, a[i].no, v, onek[v]);
        }
    }
    free(onek);
}
```

8 · Üç dil karşılaştırması. Örnek 8'in sayaç dizisi üç dilde üç biçimde kurulur: C'de `calloc` sıfırlanmış bellek verir ve `free` gerekir, C++'ta `std::vector<int> sayac(k + 1, 0)`, Python'da:

Python **H07_08_sayma_siralamasi.py · satır 17–20**

```
17      sayac = [0] * (k + 1)  # sıfırlı
18      for x in a:
19          sayac[x] += 1  # değer = indis
20          adim_sayisi += 1
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Sayaç dizisi C'de `calloc` ile sıfırlanmış bellekte (sonunda `free`), C++'ta `std::vector<int>(k + 1, 0)`, Python'da `[0] * (k + 1)` ile kurulur. Python listesinin her yuvası bir nesne başvurusudur (64 bitte 8 bayt): C'deki 4 baytlık `int` sayaç dizisinin en az iki katıdır.

9 · Test. `denetle` iki şeyi ayrı sınar: sonuç sıralı mı (kâhin: `qsort`) ve eşit anahtarlıların numaraları artan mı (kararlılık). 100 000 kayıtta yol 1 ve 2 "evet / evet", yol 3 "evet / HAYIR": **sıralı olmak kararlı olmak değildir**. **10 · Deney.** n = 10^6'da C'de K = 10^2: 1,16 ms, K = 10^6: 13,4 ms, K = 10^8: 530,5 ms; taban sıralaması aynı girdilerde 18,7, 20,3 ve 22,9 ms (Bölüm 6.4). **11 · Varyantlar. Not dağılımı:** Örnek 8 1000 notun 10'luk aralık histogramını sayaçlardan çizer ve ortancayı bulur (`ortanca (500. ve 501. not): 54, 54`). **Başlangıç konumları** (Örnek 9, yol 2): önek bir kaydırılarak tutulur, girdi soldan sağa dolaşılır (Mini-OBS v2.6 bu yolu kullanır). Negatif ya da aralık dışı anahtar: en küçük değer kadar kaydırma ya da reddetme (Hata Avcısı 4). Fikrin tarihçesi H. H. Seward'ın "dağıtım sayma" yöntemine uzanır (Seward, 1954, akt. Knuth, 1998).

4.5 Taban sıralaması (LSD)

1 · Problem. Sabit uzunlukta (d basamaklı, b tabanında) anahtarları sıralamak: üç basamaklı ondalık sayılar, 32 bitlik tamsayılar (4 bayt, b = 256). **2 · Fikir. En düşük anlamlı basamaktan** (LSD) başla: diziyi birler basamağına göre kararlı biçimde sırala, sonra onlar, sonra yüzler. Her geçiş bir sayma sıralamasıdır (k = b − 1) ve önceki geçişlerin düzenini eşitler arasında korur. Hollerith operatörlerinin kart sıralayıcılarında yaptığı budur (Bölüm 1): kartlar ceplere dağıtılır, cepler sırayla toplanır, bir sonraki sütuna geçilir (Şekil 7.9).

3 · Elle izleme. `girdi : 532 418 207 351 842 135 600 214`; birler, onlar ve yüzler geçişlerinden sonra `3. geçiş yüzler: 135 207 214 351 418 532 600 842` (Şekil 7.9).

> **TAHMİN ET · Sizce hangisi önce?**
>
> 207 ile 214'ün yüzler basamağı aynıdır (2). Üçüncü geçişten sonra hangisi önce gelir ve neden? (Yanıt: 207. İkinci geçişte onlar basamağına göre 207 (0) 214'ten (1) önce yerleşmişti; üçüncü geçiş kararlı olduğu için bu sıra korunur. Geçiş kararsız olsaydı 214, 207'den önce çıkabilirdi: Hata Avcısı 5.)

Şekil 7.9
Ondalık LSD taban sıralaması (Örnek 10): her geçişte kırmızı basamağa göre 10 cebe dağıtım ve ceplerin 0 → 9 sırayla toplanması



SÖZDE KOD · 4 · Sözde kod: LSD taban sıralaması

```
algoritma TABAN-LSD(a, n, d, b)            ▷ anahtarlar d basamaklı, b tabanında
    j ← 0'dan d − 1'e kadar her j için      ▷ j = 0: en düşük anlamlı basamak
        a'yı j. basamağa göre KARARLI-SAYMA ile sırala   ▷ k = b − 1
```

5 · Doğruluk. Değişmez (Tablo 7.7): d. geçişten sonra dizi, anahtarların en düşük d basamağına göre sıralıdır (eşitlerde önceki sıra korunur). Tümevarım: j. geçiş anahtarları j. basamağa göre sıralar; j. basamakları eşit olan iki anahtar kararlılık sayesinde önceki geçişin (alt basamakların) sırasında kalır. Böylece j + 1 alt basamağa göre sıralıdırlar. Kanıt **kararlılığa** dayanır; kararsız bir geçiş sonucu bozar (Bölüm 7, Şekil 7.14). **6 · Analiz.** d geçişin her biri Θ(n + b): **Θ(d(n + b))**; ek bellek O(n + b) (bir tampon dizi ve b sayaç). Örnek 10 adımları sayar: `adım = 1084 = 4 (2n + b - 1), n = 8, b = 256` (sekiz 32 bitlik anahtar, dört bayt geçişi; geçişler web oynatıcısında). Taban seçimi bir ödünleşimdir (n = 10^5): b = 16 ile 8 geçiş, 1 600 120 adım; b = 256 ile 4 geçiş, 801 020 adım; b = 65 536 ile 2 geçiş, 531 070 adım ama 256 KiB'lık sayaç dizisi.

Tablo 7.7
Taban sıralaması (LSD) kimlik kartı

ALGORİTMA KİMLİK KARTI	**Taban sıralaması (LSD)** *(LSD radix sort)*
Strateji	Karşılaştırmasız, basamak basamak: en düşük anlamlı basamaktan başlayıp her basamakta kararlı sayma
Girdi önkoşulu	Sabit uzunlukta anahtarlar (d basamak, taban b): ör. 32 bitlik işaretsiz tamsayı = 4 bayt; işaretli sayılarda işaret biti çevrilir (Hata Avcısı 7)
Zaman (en iyi · ortalama · en kötü)	$\Theta(d(n + b))$ · $\Theta(d(n + b))$ · $\Theta(d(n + b))$
Ek bellek	$O(n + b)$
Özellikler	Kararlı ara sıralama zorunludur · 32 bit, b = 256: 4 geçiş; Örnek 10'da n = 10^5 için 801 020 adım = $4(2n + 255)$ · taban ödünleşimi: b = 16 ile 8 geçiş, b = 65 536 ile 2 geçiş ama 256 KiB'lık sayaç
Değişmez / doğruluk	d. geçişten sonra dizi, anahtarların en düşük d basamağına göre sıralıdır (eşitlerde önceki sıra korunur)
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Çok sayıda sabit genişlikte tamsayı (öğrenci numarası, IP adresi, 32/64 bitlik anahtar)
Ne zaman kullanılmaz	n küçükken (sayaç dizisi ve geçiş maliyeti baskın); anahtarlar değişken uzunlukta ya da basamak yapısı yoksa

7 · C gerçekleştirimi. (Örnek 10; `gecis` bir basamakta kararlı saymadır; `taban_siralamasi` `kaydir` = 0, 8, 16, 24 için `gecis`'i çağırır ve tampon ile dizinin rollerini her geçişte değiştirir, 72–76. satırlar):

C `c/H07_10_taban_siralamasi.c · satır 40–59`

```c
40 static void gecis(const uint32_t *a, uint32_t *b, int n, int kaydir,
41                   uint32_t maske, int *onek)
42 {
43     for (uint32_t v = 0; v <= maske; v++) {
44         onek[v] = 0;
45     }
46     for (int i = 0; i < n; i++) {
47         onek[(a[i] >> kaydir) & maske]++;
48         adim_sayisi++;
49     }
50     for (uint32_t v = 1; v <= maske; v++) {
51         onek[v] += onek[v - 1];
52         adim_sayisi++;
53     }
54     for (int i = n - 1; i >= 0; i--) {
55         uint32_t v = (a[i] >> kaydir) & maske;
56         b[--onek[v]] = a[i];
57         adim_sayisi++;
58     }
59 }
```

8 · Üç dil karşılaştırması. Basamak üç dilde `(x >> kaydir) & maske` ile alınır; C ve C++'ta anahtar işaretsiz `uint32_t`'dir; C++ döngüsü C'ninkiyle aynıdır (`b[--onek[v]] = a[i]`), Python'da `--` olmadığı için iki deyim gerekir:

Python **H07_10_taban_siralamasi.py · satır 42–45**

```python
    for i in range(n - 1, -1, -1):
        v = (a[i] >> kaydir) & maske
        onek[v] -= 1
        b[onek[v]] = a[i]
```

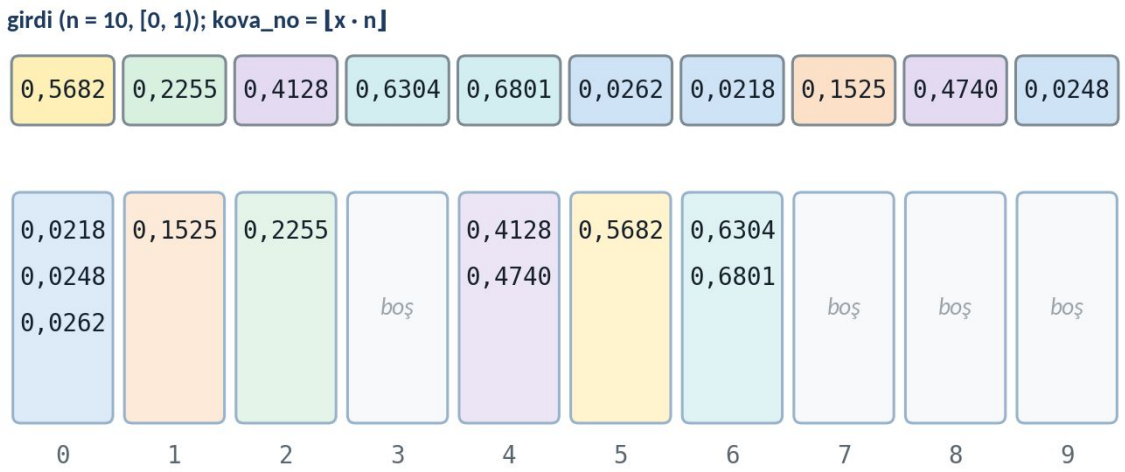
> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Bayt C/C++'ta işaretsiz `uint32_t` üzerinde `(x >> kaydir) & maske` ile alınır; Python tamsayısı sınırsızdır ve negatif sayıyı sağa kaydırınca işaret biti sonsuza dek genişler — işaretli sayılar ayrıca ele alınmalıdır (Hata Avcısı 7). C'de tek ifadedeki iki `rastgele_sayi()` çağrısının sırası belirtilmemiştir; bu yüzden iki ayrı deyim yazılır (Python soldan sağa değerlendirir). On altılık yazım: `%08X`, `std::hex + std::setw(8) + std::setfill('0')`, `f"{x:08X}"`.

9 · Test. Her ölçekte sonuç `qsort` kâhiniyle karşılaştırılır (“evet”); adım sayısı formülle birebir aynıdır. **10 · Deney.** Süre anahtar aralığından neredeyse bağımsızdır: n = 10^6'da K = 10^2 … 2^{32} için C'de 18,7–28,6 ms (hep dört geçiş; Bölüm 6.4). **11 · Varyantlar.** Geçiş sayısını en büyük anahtardan seçmek (deneydeki “taban, geçiş sayısı K'ya göre”; Mini-OBS v2.6); işaretli tamsayılarda işaret bitini çevirmek (Hata Avcısı 7); en yüksek anlamlı basamaktan başlayan MSD taban sıralaması ve dizgiler (İleri Bakış).

4.6 Kova sıralaması

1 · Problem. [0, 1) aralığında yaklaşık düzgün dağılmış n gerçel sayıyı sıralamak. **2 · Fikir.** Aralığı n eşit kovaya böl; x'i ⌊x · n⌋ numaralı kovaya koy, her kovayı araya eklemeyle sırala, kovaları sırayla birleştir. Dağılım düzgünse kova başına ortalama bir eleman düşer. Örnek 11 kovaları ayrı listeler yerine tek dizide ardışık dilimler olarak tutar: kova boyları sayılır ve önek toplamla başlangıçlar bulunur (sayma sıralamasının fikri). **3 · Elle izleme.** Örnek 11'in on değeri (Şekil 7.10): `karşılaştırma = 5, boş kova = 4, en dolu kova = 3`.

Şekil 7.10
Kova sıralaması (Örnek 11): on değer on kovaya dağıtılır; kova içi araya ekleme, kovalar sırayla birleştirilir



kova içi araya ekleme sıralanır, kovalar sırayla birleştirilir · 4 boş kova, en dolu kova 3 eleman

4 · Sözde kod. KOVA(a, n): her x için kova[⌊x·n⌋]'a ekle; her kovayı ARAYA-EKLEME ile sırala; kovaları 0'dan n − 1'e birleştir. **5 · Doğruluk.** Değişmez (Tablo 7.8): Kova v'deki her değer [v/n, (v + 1)/n)

aralığındadır; kovalar sırayla birleştirildiğinde, her kova sıralıysa tüm dizi sıralıdır. **6 · Analiz.** Beklenen Θ(n): düzgün dağılımda eleman başına 0,453–0,467 karşılaştırma, boş kova oranı 0,369. En kötü durum Θ(n^2): bütün değerler tek kovaya düşerse araya ekleme karesel çalışır. n = 4000'de eleman başına karşılaştırma düzgün dağılımda 0,455, u^4 ile çarpıtılmış dağılımda 18,5, tek kovada 1009,0.

Tablo 7.8
Kova sıralaması kimlik kartı

ALGORİTMA KİMLİK KARTI **Kova sıralaması** *(bucket sort)*	
Strateji	Dağıt–sırala–birleştir: [0, 1) aralığı n eşit kovaya bölünür, kova içleri araya eklemeyle sıralanır
Girdi önkoşulu	Değerler [0, 1) aralığında ve yaklaşık düzgün dağılmış
Zaman (en iyi · ortalama · en kötü)	Θ(n) · Θ(n) (düzgün dağılım) · Θ(n^2)
Ek bellek	O(n)
Özellikler	Örnek 11: düzgün dağılımda eleman başına 0,45–0,47 karşılaştırma ve boş kova oranı 0,369; u^4 dağılımında n = 4000'de 18,5, tüm değerler tek kovadaysa 1009 karşılaştırma/eleman
Değişmez / doğruluk	Kova v'deki her değer [v/n, (v + 1)/n) aralığındadır; kovalar sırayla birleştirildiğinde, her kova sıralıysa tüm dizi sıralıdır
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Dağılımı bilinen, [0, 1) aralığına ölçeklenmiş ondalık veriler
Ne zaman kullanılmaz	Dağılım bilinmiyor ya da çarpıksa; tamsayı anahtarlarda sayma ya da taban sıralaması daha uygundur

7 · C gerçekleştirimi. (Örnek 11, dağıtım bölümü; 83. satırda her kova `araya_ekleme(b, bas[v], bas[v + 1])` ile sıralanır):

c/H07_11_kova_siralamasi.c · satır 57–68

```c
    for (int i = 0; i < n; i++) {
        bas[kova_no(a[i], n) + 1]++;       /* kova boyları */
    }
    for (int v = 1; v <= n; v++) {
        bas[v] += bas[v - 1];              /* kova v: b[bas[v]..bas[v+1]) */
    }
    for (int v = 0; v < n; v++) {
        yer[v] = bas[v];
    }
    for (int i = 0; i < n; i++) {          /* dağıt (kararlı) */
        b[yer[kova_no(a[i], n)]++] = a[i];
    }
```

8 · Üç dil karşılaştırması. Kova numarası C'de `(int)(x * n)`, Python'da `int(x * n)`; Python'da sonuç `a[:] = b` ile çağıranın listesine yazılır (`a = b` yalnızca yerel adı bağlardı; Ek A.11). **9 · Test.** Kâhin her ölçekte "evet"; ondalıklar üç dilde aynı basamakla yazıldığı için çıktılar bayt bayt aynıdır. **10 · Deney.** Yukarıdaki dağılım tablosu (Örnek 11, 3. bölüm). **11 · Varyantlar.** Genel [min, max] aralığına ölçekleme; kova içinde başka sıralama; dağılımı bilinmeyen veride taban sıralaması daha güvenlidir.

4.7 Bütüncül karşılaştırma (H5–H7) ve algoritma seçimi

Tablo 7.9 haftaların kimlik kartlarını ve bu haftanın ölçümlerini birleştirir. Karmaşıklıklar kartlardan, süreler Haftanın Deneyi'nden (C -O2, n = 10^6 rastgele girdi, işlemci zamanı) gelir; "≈" ölçülmemiş, n^2 modeliyle kestirilmiş değerdir.

Tablo 7.9
H5–H7 sıralama algoritmalarının bütüncül karşılaştırması (k: anahtar aralığı, d: basamak, b: taban)

Algoritma	En iyi	Ortalama	En kötü	Ek bellek	Kararlı	Yerinde	Uyarl.	10^6 (C)
seçmeli (H5)	Θ(n^2)	Θ(n^2)	Θ(n^2)	O(1)	hayır	evet	hayır	≈ 21,5 dk
kabarcık (erken çıkışlı) (H5)	O(n)	Θ(n^2)	Θ(n^2)	O(1)	evet	evet	evet	—
araya ekleme (H5)	O(n)	Θ(n^2)	Θ(n^2)	O(1)	evet	evet	evet	≈ 1,8 dk
Shell (H5)	O(n log n)	aralığa bağlı	O(n√n)	O(1)	hayır	evet	evet	—
birleştirmeli (H6)	Θ(n log n)	Θ(n log n)	Θ(n log n)	O(n)	evet	hayır	hayır	120,9 ms
hızlı (H6)	Θ(n log n)	Θ(n log n)	Θ(n^2)	O(log n) yığın	hayır	evet	hayır	101,9 ms
öbek (H7)	Θ(n log n)	Θ(n log n)	Θ(n log n)	O(1)	hayır	evet	hayır	207,2 ms
sayma (H7)	Θ(n + k)	Θ(n + k)	Θ(n + k)	O(n + k)	evet	hayır	hayır	18,7 ms
taban (LSD) (H7)	Θ(d(n + b))	Θ(d(n + b))	Θ(d(n + b))	O(n + b)	evet	hayır	hayır	20,3 ms
kova (H7)	Θ(n)	Θ(n)	Θ(n^2)	O(n)	evet	hayır	hayır	—
kütüphane: C `qsort`	—	—	—	—	belirtilmez	—	—	142,0 ms

Seçim, verinin **profiline** bağlıdır: boyut, anahtar türü ve aralığı, girdinin düzeni, kararlılık gereksinimi ve bellek. Şekil 7.11 bu soruları bir karar akışına dizer (aşağıda); her dalın gerekçesi bu haftanın ölçümlerindedir:

- **Küçük aralıklı tamsayı → sayma.** n = 10^6, K = 10^2: sayma 1,16 ms, `qsort` 95,3 ms. Notlar, yaşlar, baytlar.
- **Geniş aralıklı tamsayı → taban.** K = 2^{32}: taban 28,6 ms, `qsort` 139,6 ms; sayma uygulanamaz.
- **Yalnız ilk k → öbek (ilk-k).** n = 10^5, k = 10: 100 205 karşılaştırma; tam sıralama ölçeği n $\log_2$ n = 1 660 964 (Bölüm 5.2).
- **Neredeyse sıralı ya da küçük → araya ekleme.** n = 10^4 sıralı girdide 17,0 µs (rastgelede 9,93 ms).
- **Kararlılık gerekli → birleştirmeli** (C++ `std::stable_sort`, Python `sorted`/`list.sort`; Bölüm 8). Öbek, hızlı ve seçmeli kararsızdır.
- **Ek bellek yok ve en kötü durum güvencesi → öbek sıralaması** (gömülü sistem, gerçek zamanlı sınır).
- **Hiçbiri → kütüphane sıralaması**, ama önce ölçün: verinin gerçek profili çoğu zaman sezgiden farklıdır (Bölüm 6 ve 9).

Şekil 7.11
Sıralama algoritması seçim rehberi: soruların sırası karşılaştırmasız yöntemlerin koşullarıyla başlar; hiçbiri uymazsa kütüphane sıralaması (önce ölçün)



5. Çözümlü Örnekler

Örnek merdiveninin amiral olmayan basamakları burada. Hepsinin C, C++ ve Python sürümleri aynı çıktıyı verir (`kodlar/esdeger_test.py`); C++ ve Python çekirdek fonksiyonları Ek A'dadır. Örnek 13 (iki öbekle akan ortanca) Merak Kutusu'ndadır (Bölüm 13).

5.1 Öbek özelliği denetimi (Örnek 2)

Bir dizinin öbek olup olmadığını bulmak için her i > 0 için `a[ebeveyn(i)] < a[i]` ihlalini aramak yeter (`c/H07_02_obek_mi.c · satır 21–30`); ilk ihlalde durulur. Bu fonksiyon haftanın bütün öbek programlarında **kâhin** olarak kullanılır (her işlemden sonra "öbek mi?"). Örnek 2'nin iki satırı bir test dersi verir: `artan sıralı (n = 10): öbek: HAYIR (i = 1: a[0] = 45 < a[1] = 52), karşılaştırma = 1`, ama `son yaprakta ihlal (n = 10): öbek: HAYIR (i = 9: a[4] = 84 < a[9] = 85), karşılaştırma = 9`. Tek bir yapraktaki ihlal ancak son karşılaştırmada görülür; yalnızca kökü ya da ilk düzeyleri denetleyen bir test onu kaçırır. Hata Avcısı 1(b)'deki bozuk öbeklerin çoğu da böyle derinde saklanır.

5.2 Öncelik kuyruğuyla ilk-k (Örnek 6)

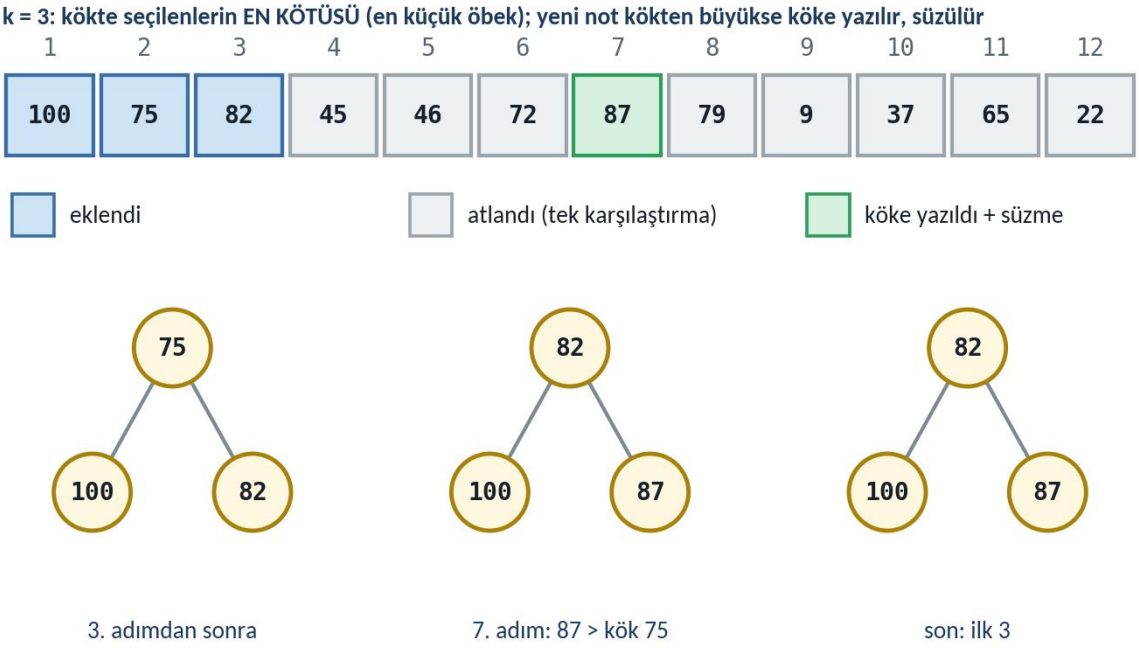
n öğrencinin en yüksek notlu k'sını bulmak için bütün listeyi sıralamak gerekmez (O(n log n)). **Boyutu k olan en küçük öbek** tutulur: kökte şimdiye kadar seçilenlerin **en kötüsü** durur. Yeni not kökten iyiyse kök atılır, yeni not köke yazılıp aşağı süzülür; değilse tek karşılaştırmayla atlanır. Maliyet O(n log k), bellek O(k) (Şekil 7.12). Eşit notlarda küçük numara önce gelir; ölçüt böylece tamdır ve sonuç tek tanımlıdır (kâhinle karşılaştırılabilir).

c/H07_06_ilk_k.c · satır 65–81

```c
static int ilk_k(const Ogrenci *a, int n, int k, Ogrenci *h, int izle)
{
    int boyut = 0;
    for (int i = 0; i < n; i++) {
        const char *ne;
        if (boyut < k) {
            h[boyut++] = a[i];
            yukari_suz(h, boyut - 1);
            ne = "eklendi";
        } else if (kotu_mu(h[0], a[i])) {
            h[0] = a[i];                    /* en kötüyü at */
            asagi_suz(h, k, 0);
            degisim_sayisi++;
            ne = "köke yazıldı";
        } else {
            ne = "atlandı";
        }
```

Döngüden sonra (91–94. satırlar) öbek yerinde sıralanır, böylece h[0] en iyi olur: `ilk 3: 2400001 (100) 2400007 (87) 2400003 (82)`. n = 10^5 rastgele notta k = 1 için 99 999, k = 10 için 100 205, k = 1 000 için 185 902 karşılaştırma yapılır; tam sıralamanın ölçeği n $\log_2$ n = 1 660 964'tür. Rastgele girdide çoğu not ilk karşılaştırmada elenir (karş./n ≈ 1); **artan** puanlı girdide her not köke yazılır ve karş./n ≈ 2 $\log_2$ k olur (k = 1000: 18,926 ve 19,932).

Şekil 7.12
İlk-k, k = 3 (Örnek 6): her not için "eklendi / atlandı / köke yazıldı"; kök seçilenlerin en kötüsü



5.3 k sıralı listenin birleştirilmesi (Örnek 7)

H6'daki iki yollu birleştirmenin genellemesi: k şubenin sıralı not listesi tek sıralı listede birleştirilecek. Her listenin baş elemanı bir **en küçük öbekte** tutulur (öğe: değer, liste, konum). Her adımda kök çıktıya

yazılır, aynı listenin sıradaki elemanı köke konur ve aşağı süzülür; liste bittiyse son öğe köke taşınır. N eleman için O(N log k). Beş listeli örnekte (D boş) çıktı `başlangıç öbeği: 38/C 50/B 45/A 66/E` satırını verir (değer/liste). Eşit değerlerde küçük liste numarası önce gelir; birleştirme böylece kararlıdır.

c/H07_07_k_yollu_birlestirme.c · satır 97–107

```c
    while (boyut > 0) {
        Oge en = h[0];
        cikti[m++] = en.deger;
        if (en.konum + 1 < uzunluk[en.liste]) {
            int s = en.konum + 1;
            h[0] = (Oge){liste[en.liste][s], en.liste, s};
        } else {
            boyut--;
            h[0] = h[boyut];              /* liste bitti */
        }
        asagi_suz(h, boyut, 0);
```

Beş listede `eleman = 11, karşılaştırma = 23, sıralı mı? evet`. N = 16 384 elemanda k = 2 için öbek 16 281 karşılaştırma yapar (tarama ile aynı), k = 64 için 147 356 (N başına 8,994; $\log_2$ k = 6,000); her adımda k baş elemanı tarayan yöntem ise 1 007 019 (N başına 61,464). Dış bellekte sıralamada (bellek yetmeyince parçaları ayrı sıralayıp birleştirmek) bu yöntem kullanılır.

5.4 Sıralama karşılaştırma deneyi: sayaçlar (Örnek 12)

Örnek 12 ara projenin omurgasıdır: sekiz algoritmayı aynı dört girdi türünde (rastgele, sıralı, ters sıralı, çok tekrarlı) çalıştırır, karşılaştırma, yazma ve sayaç işlemlerini ayrı sayar ve her sonucu kütüphane sıralamasıyla karşılaştırır (`kâhinle aynı mı (4 girdi türü): 32 / 32 evet`). Süre ölçülmez: sayaçlar makineden bağımsızdır ve kuramla doğrudan karşılaştırılır. Süreler Haftanın Deneyi'nin sayaçsız programlarıyla ölçülür (Bölüm 6). Aşağıda n = 1000 ve 2000 için toplam işin oranı:

Rastgele girdide oranlar: seçmeli 3,994, araya ekleme 3,891, birleştirmeli 2,211, hızlı 2,164, öbek 2,235, sayma 1,002, taban 1,887.

Karesel algoritmalarda oran 4'e, n log n algoritmalarda 2,2'ye yakındır; araya ekleme sıralı girdide doğrusaldır (2,0). Sayma sıralamasında oran 1'dir, çünkü iş k = 10^6'lık sayaç taramasıyla baskındır; anahtarları 0 … 9 olan çok tekrarlı girdide k küçüktür ve oran 2'ye çıkar. Taban sıralamasında 1,89: dört geçişin her biri 2n + 255 adım, sabit 255 küçük n'de oranı 2'nin altına çeker.

6. Haftanın Deneyi: Sekiz Sıralama, Dört Girdi, Anahtar Aralığı

Soru. Sekiz sıralama algoritmasının rastgele, sıralı, ters sıralı ve çok tekrarlı girdilerde karşılaştırılması; sayma ve taban sıralamasının anahtar aralığına duyarlılığı (ara proje raporunun omurgası). Üç deney vardır: `karsilastir` (8 algoritma × 4 girdi × n), `aralik` (n sabit, anahtarlar [0, K) aralığında, K = 10^2 … 2^{32}) ve `kurma` (aşağıdan yukarıya ve tek tek ekleyerek kurma × rastgele ve artan girdi × n = 10^3 … 10^7). Programlar `kodlar/deney/` altındadır; algoritmalar Örnek 12'ninkilerle aynıdır, ama sayaç içermez.

6.1 Ölçüm düzeni

OLCUM_ORTAMI.TXT (SEÇİLİ SATIRLAR)

```
İşlemci        : Intel(R) Xeon(R) Processor @ 2.80GHz
Çekirdek sayısı: 2 (sanal makine; ölçüm sırasında başka süreçler de çalışıyordu)
Yük ortalaması : önce 4.02 2.74 2.25 · sonra 2.17 3.08 3.22 (/proc/loadavg: 1, 5, 15
dk)
C derleyicisi  : gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Python         : Python 3.11.15
Bayraklar      : C/C++ -std=c17 / -std=c++17 -Wall -Wextra -pedantic -O2 (C ayrıca -
O0)
Yöntem         : 1 ısınma + 5 tekrar (büyük n'de 3; doğrulama ölçümlerinde 1),
ortanca; işlemci zamanı en_cok/en_az > 1,5 ise en çok 2 kez yineleme
Saat           : duvar saati C timespec_get, C++ steady_clock, Python
time.perf_counter; işlemci zamanı C/C++ clock() (1 µs çözünürlük), Python
time.process_time
Veri türü      : 32 bitlik işaretsiz anahtarlar (C/C++ uint32_t)
```

Ölçüm sırasında makinede başka işler de çalışıyordu (yük ortalaması yukarıda). Duvar saati sürecin bekleme süresini de içerdiğinden şişti: duvar/işlemci oranının ortancası C 1,25, C++ 1,15, Python 1,39. Bu yüzden çözümleme ve grafiklerde **işlemci zamanı ortancası** kullanıldı (duvar saati CSV'de duruyor). Bütün ızgara iki bağımsız oturumda ölçüldü ve her ölçüm için küçük olan alındı: başka süreçlerin etkisi süreyi yalnızca artırabilir. Toplam 511 ölçümün 507'sinde sonuç kâhinle aynıydı; 4 ölçüm “uygulanamaz”dı (sayma sıralamasında çok büyük K), yanlış sonuç yoktu; yayılımı büyük 46 ölçüm yinelendi.

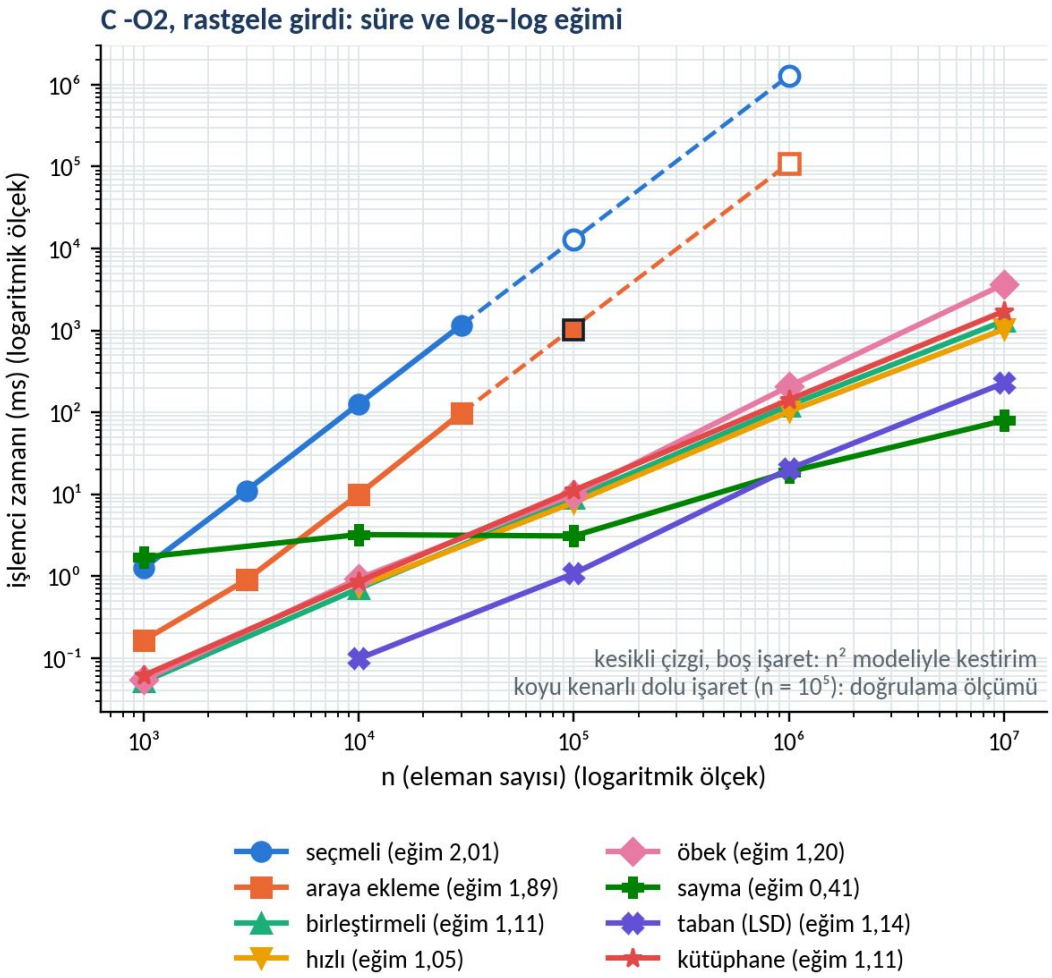
6.2 Sonuçlar

Tablo 7.10

$n = 10^6$ için işlemci zamanı ortancası, bu makinede (≈: ölçülmedi; $n = 3{\cdot}10^4$ ölçümünden n^2 modeliyle, araya eklemenin sıralı girdisinde n modeliyle kestirim)

Algoritma (C -O2)	rastgele	sıralı	ters sıralı	çok tekrarlı
seçmeli	≈ 21,5 dk	≈ 21,2 dk	≈ 21,3 dk	≈ 20,7 dk
araya ekleme	≈ 1,8 dk	≈ 1,53 ms	≈ 4,2 dk	≈ 1,6 dk
birleştirmeli	120,9 ms	26,7 ms	30,8 ms	64,4 ms
hızlı	101,9 ms	18,0 ms	18,9 ms	33,1 ms
öbek	207,2 ms	66,5 ms	69,3 ms	69,2 ms
sayma	18,7 ms	4,85 ms	4,32 ms	1,20 ms
taban (LSD)	20,3 ms	30,6 ms	30,1 ms	15,5 ms
kütüphane (`qsort`)	142,0 ms	31,9 ms	41,7 ms	75,6 ms

Şekil 7.13
C -O2, rastgele girdi: süre ve log–log eğimleri (iki eksen de logaritmik; kesikli çizgi ve boş işaret: kestirim)



6.3 Kuramla karşılaştırma

Log–log eğimi büyüme sınıfını verir (Şekil 7.13): seçmeli 2,01, araya ekleme 1,89 (≈ 2, karesel); birleştirmeli 1,11, hızlı 1,05, öbek 1,20, kütüphane 1,11 (n log n'in eğimi bu aralıkta 1'in biraz üstündedir); taban 1,14. “10 kat oranı” t(n_max)/t(n_max/10) aynı şeyi söyler: seçmeli 105,3, birleştirmeli 10,8, hızlı 10,1, öbek 17,7. Sayma sıralamasının eğimi 0,41'dir: süre K = 10^6'lık sayaç taramasıyla baskındır, n büyüdükçe bu sabit iş daha çok elemana bölünür. Ölçülmeyen büyüklükler bir modelle kestirildi ve model ölçülebilen bir noktada sınandı (Tablo 7.11).

Tablo 7.11
Kestirimlerin sınanması: model, ölçülen bir noktadan ötekini kestirir (hata = kestirim / ölçülen − 1)

Dil	Algoritma	n	Kestirim	Ölçülen	Hata	Model
C -O2	araya ekleme	100 000	1,09 s	1,01 s	%8,1	n^2 (taban n = 30 000)
Python	seçmeli	10 000	1,86 s	2,12 s	%−12,5	n^2 (taban n = 3 000)
Python	araya ekleme	10 000	1,73 s	2,08 s	%−16,9	n^2 (taban n = 3 000)
C -O2	birleştirmeli	10 000 000	1,41 s	1,31 s	%8,0	n log n ($10^6 \rightarrow 10^7$)
C -O2	hızlı	10 000 000	1,19 s	1,03 s	%15,0	n log n ($10^6 \rightarrow 10^7$)
C -O2	öbek	10 000 000	2,42 s	3,67 s	%−34,1	n log n ($10^6 \rightarrow 10^7$)

Dil	Algoritma	n	Kestirim	Ölçülen	Hata	Model
C -O2	sayma	10 000 000	175,2 ms	79,3 ms	%120,8	$\alpha n + \beta$ (10^5, $10^6 \rightarrow 10^7$)
C -O2	taban	10 000 000	202,9 ms	227,9 ms	%−11,0	n ($10^6 \rightarrow 10^7$)
C -O2	kütüphane	10 000 000	1,66 s	1,71 s	%−3,3	n log n ($10^6 \rightarrow 10^7$)

Karesellerde model iyi tutar; n log n algoritmalarda ve taban sıralamasının n modelinde 10^6'dan 10^7'ye kestirim %15'in içindedir, **öbek sıralaması dışında** (%−34,1): dizi önbelleğe sığmayınca uzak erişimlerin bedeli artar (Bölüm 3.4). Sayma sıralamasında $\alpha n + \beta$ modeli 10^7'yi fazla kestirdi (%120,8): K sabitken eleman başına süre n büyüdükçe azalıyor. Kuram büyüme sınıfını verir; sabitleri ve donanım etkilerini ölçüm gösterir.

6.4 Anahtar aralığına duyarlılık

Tablo 7.12
Anahtar aralığı [0, K) ile süre (işlemci zamanı ortancası, bu makinede)

Algoritma (C, n = 10^6)	K = 10^2	K = 10^4	K = 10^6	K = 10^7	K = 10^8	K = 2^{32}
sayma	1,16 ms	1,37 ms	13,4 ms	75,2 ms	530,5 ms	uygulanamaz
taban (4 geçiş)	18,7 ms	28,6 ms	20,3 ms	24,7 ms	22,9 ms	28,6 ms
taban (geçiş K'ya göre)	7,36 ms	20,0 ms	25,5 ms	21,4 ms	19,5 ms	28,7 ms
hızlı	45,5 ms	71,6 ms	107,0 ms	93,9 ms	109,0 ms	90,8 ms
kütüphane	95,3 ms	121,5 ms	136,9 ms	151,8 ms	150,3 ms	139,6 ms

K küçükken sayma sıralaması en hızlıdır; K ≈ n civarında da taban sıralamasıyla aynı mertebededir, K ≫ n olunca sayaç dizisinin taranması ve önbelleğe sığmaması baskın olur (sayma / taban süre oranı: K = 10^2 için 0,06, 10^6 için 0,66, 10^8 için 23,20). Taban sıralamasının süresi K'dan neredeyse bağımsızdır; geçiş sayısını en büyük anahtardan seçen sürüm küçük K'da gereksiz geçişleri atlar.

6.5 Öbek kurma ve diller

Aşağıdan yukarıya kurmada takas/n hiçbir n'de 1'i geçmez (artan girdide en çok 1,000, rastgele girdide 0,744–0,754); tek tek eklemeyle artan girdide takas/n 7,987'den (n = 10^3) 21,322'ye (n = 10^7) çıkar: $\log_2$ n ile büyür. Eleman başına süre ise n büyüdükçe artar (aşağıdan, rastgele: 5,0 → 10,5 ns): sayaç doğrusal, süre tam doğrusal değil. Raporunuz için iyi bir "kuram ve ölçüm farkı" örneği.

Diller (n = 10^5 rastgele, işlemci zamanı): C++ elle yazılan algoritmalarda C'ye çok yakındır (öbek 0,99, sayma 1,04); kütüphane satırında ise C++'ın `std::sort`'u C'nin `qsort`'unun 0,55 katı sürede biter (satır içi karşılaştırma ve fonksiyon işaretçisi farkı). Python yorumlayıcı yükü elle yazılan algoritmalarda büyüktür (öbek 51,4, taban 166,3 kat), ama Python'un kendi `list.sort`'u (C'de yazılmış Timsort) `qsort`'un yalnızca 2,2 katıdır: **Python'da kütüphaneyi kullanın.**

6.6 Yorum, sınırlılıklar ve rapor nasıl yazılır?

- **Karesel ve n log n ayrımı ölçümle doğrulanır;** n = 10^6'da seçmeli sıralama kestirimi kütüphanenin ≈ 9 087 katıdır.
- **Doğrusal sıralamalar** (K = 10^6) kütüphaneden ≈ 7 kat hızlıdır; sayma ile taban arasındaki fark gürültü düzeyindedir.

- **Öbek sıralaması** hızlı sıralamanın ≈ 2 katı sürer (oturuma göre 1,75–2,52); güvencesi için seçilir, hızı için değil.
- **Sınırlılıklar:** paylaşımlı, 2 çekirdekli sanal makine; tek veri türü (32 bit) ve tek tohum (42); aynı ölçümün iki oturum arasındaki oranı ortancada ≈ 1,1, en kötü ≈ 2,5: %10–15'in altındaki farkları yorumlamayın. Kestirimler model varsayımıdır.

> MİNİ-OBS 2 PROJESİ · **Rapor nasıl yazılır? Bir bulgu paragrafı örneği**
>
> *"$n = 10^6$ rastgele 32 bitlik anahtarda öbek sıralaması 207,2 ms, hızlı sıralama 101,9 ms sürdü (C -O2, işlemci zamanı, 1 ısınma + 5 tekrarın ortancası; iki oturumun küçüğü; Tablo 3). İkisinin karşılaştırma sayıları aynı mertebededir (Tablo 7: n = 2000'de 37 717 ve 33 429), dolayısıyla fark karşılaştırma sayısından değil, bellek erişim düzeninden kaynaklanıyor olabilir (ölçülmedi). Oran oturuma göre 1,75 ile 2,52 arasında değişti; bu yüzden 'yaklaşık iki kat' diyoruz."*
>
> Bu paragrafta bir raporda olması gereken her şey vardır: **sayı ve birim**, **düzen** (derleme, süre ölçütü, tekrar, ortanca), **kaynak tablo**, kuramla bağ (sayaç), **neden** için açıkça işaretlenmiş bir olasılık ("ölçülmedi") ve **belirsizlik**. Şablonun bölümleri: Özet → Amaç ve sorular → Yöntem (ortam, algoritmalar, girdiler, ölçüm düzeni, doğruluk) → Sonuçlar → Tartışma (kuramla tutarlılık, seçim rehberi) → Geçerlilik tehditleri → Sonuç → Ekler (Bölüm 10.5).

Kendi makinenizde. `cd kodlar/deney && python3 h07_olcum_calistir.py && python3 h07_cozumle.py` bütün ölçümleri yapar ve özet CSV'leri üretir (bu makinede işlemci zamanı toplamı ≈ 3,8 dk). `sonuclar/h07_deney_karsilastirma.csv` dosyanızı web sayfasındaki **sıralama karşılaştırma panosuna** yükleyerek ısı haritasını ve eğimleri kendi ölçümlerinizle görebilirsiniz. Sizin süreleriniz farklı olacaktır; büyüme sınıfları, sayaçlar ve karşılaştırmaların yönü aynı kalmalıdır.

7. Hata Avcısı

Her satırın hatalı ve düzeltilmiş programı `kodlar/hata_avcisi/` altındadır; belirtiler bu makinede gözlenen çıktılardır (`hata_avcisi/ciktilar/`). Hepsi uyarısız derlenir: hatalar mantıksaldır (Hata Avcısı 4'ün C sürümü tanımsız davranıştır).

Tablo 7.13
Hata Avcısı: hata, belirti, neden ve çözüm

#	Hata	Belirti (bu makinede)	Neden	Çözüm
1	0 ve 1 tabanlı indis formüllerini karıştırmak	çocuklar 2i, 2i + 1: `azalan mı? HAYIR`; ebeveyn i / 2: 10 000 rastgele dizide 408 bozuk öbek (çoğu girdide görünmez)	1 tabanlı kitap formülleri 0 tabanlı diziye taşındı; kökün sol çocuğu kendisi oldu	(i − 1)/2, 2i + 1, 2i + 2 birlikte; tek yerde fonksiyon; kâhinli rastgele test
2	Aşağı süzmede büyük değil ilk çocuğu seçmek	`öbek mi? HAYIR; sıralandı : 45 63 52 79 90 71 92 84 88 95`	Küçük çocuk köke çıkınca büyük kardeşin ebeveyni olur	Önce büyük çocuğu bul, yalnız onunla karşılaştır
3	Öbek kurmayı i = 0'dan başlatmak	`öbek mi? HAYIR; kök = 63 (en büyük 95 olmalı)`	Kök süzülürken alt ağaçları henüz öbek değil	i = ⌊n/2⌋ − 1'den 0'a
4	Sayma sıralamasında aralık dışı anahtar (−1, 105)	C (tanımsız davranış; bu çalıştırmada): `double free or corruption (out)`, ASan: heap-buffer-overflow. Python: −1 sessizce 100 sayılır, 105 → `IndexError`	`sayac[a[i]]` için 0 ≤ a[i] ≤ k denetlenmiyor; Python'da negatif indis sondan sayar	Her anahtarı sayaçtan önce denetle ya da en küçük değer kadar kaydır
5	Taban sıralamasında kararsız ara geçiş	`3. geçiş yüzler : 135 214 207 351 418 532 600 842` → `sıralı mı? HAYIR`	Bitiş konumlu önekle soldan sağa yerleştirme eşitleri ters çevirir	Sağdan sola yerleştir (ya da başlangıç önekiyle soldan sağa)

#	Hata	Belirti (bu makinede)	Neden	Çözüm
6	Python heapq'nun en küçük öbek olduğunu unutmak	`en yüksek 3 not (sanılan): [45, 52, 63]`	`heap[0]` en küçüktür	−x hilesi ya da `heapq.nlargest`
7	İşaretli sayıları işaretsiz gibi taban sıralamasına sokmak	`sıralandı: 0 3 17 42 2147483647 -2147483648 -5 -1`	İkiye tümleyende negatiflerin üst baytı ≥ 0x80; Python'da sağa kaydırma işareti genişletir	C: `(uint32_t)x ^ 0x80000000`; Python: x + 2^{31}

Şekil 7.14
Hata Avcısı 5: kararlı (solda) ve kararsız (sağda) geçişler; kırmızı hücreler doğru geçişten ayrılan konumlar



2. geçişte onlar basamağı eşit 600 ve 207 ters döndü; LSD önceki geçişin düzenine güvenir · son dizi sıralı mı? kararlı: evet, kararsız: HAYIR

Hata Avcısı 5'e yakından bakış. Hatalı geçiş bitiş konumlu önek dizisiyle girdiyi soldan sağa dolaşır; doğrusu Bölüm 4.5'teki `gecis` gibi sağdan sola dolaşmaktır. Tek satırlık fark (Şekil 7.14): her geçiş "sıralı" görünür, ama eşit basamaklıların sırası ters döndüğü için sonraki geçişlerin dayandığı düzen kaybolur.

C `hata_avcisi/H07_HA5_kararsiz_gecis.c · satır 23–25`

```c
    for (int i = 0; i < n; i++) {      /* HATA: sağdan sola olmalı */
        b[--onek[a[i] / bolen % 10]] = a[i];
    }
```

8. Dil Penceresi: C · C++ · Python

Tablo 7.14
Dil karşılıkları: öncelik kuyruğu, sayaç dizisi, bit işlemleri ve kararlılık

Öğe	C	C++	Python
Öncelik kuyruğu	yok; öbek elle yazılır (Kavram 7)	`std::priority_queue<T>`: varsayılan **en büyük**	`heapq` (liste üzerinde): **en küçük**
En küçük / en büyük	karşılaştırma fonksiyonu çevrilir	`std::greater<T>` ile en küçük	−x hilesi; `nlargest`, `nsmallest`

Öğe	C	C++	Python
Öbek kurma / süzme	elle: `asagi_suz`, `yukari_suz`	`std::make_heap`, `push_heap`, `pop_heap`, `sort_heap`	`heapify`, `heappush`, `heappop`
Sayaç dizisi	`calloc(k + 1, sizeof *s)` + `free`	`std::vector<int>(k + 1, 0)`	`[0] * (k + 1)` (yuva 8 bayt)
Bayt alma	`(x >> (8 * d)) & 0xFF` (`uint32_t`)	aynı (`std::uint32_t`)	aynı, ama negatifte işaret genişler
Kararlı sıralama	standartta yok (`qsort` belirtmez)	`std::stable_sort` (kararlı), `std::sort` (değil)	`sorted`, `list.sort` (kararlı, Timsort)
Eşitlikte ikinci ölçüt	karşılaştırıcıda elle	`std::tie(a, b) < …`, demet karşılaştırma	`key=lambda r: (r.not_, r.no)`

C++. `std::priority_queue` varsayılan olarak en büyük öbektir; üçüncü şablon parametresi `std::greater<int>` en küçük öbek yapar. İlk-k için boyutu k olan **en küçük** öbek gerekir (Örnek 6'nın fikri): Kavram 1 (`kavram/H07_K01_priority_queue.cpp`, 42–52. satırlar) `ilk.size() < k` iken ekler, değilse yeni not kökten (`ilk.top()`) büyükse kökü atıp ekler:

ÇIKTI

```
$ ./H07_K01_priority_queue
en büyük öbek (top, pop): 95 90 88 84 71 63 52 45
en küçük öbek (top, pop): 45 52 63 71 84 88 90 95
ilk 3 (küçükten büyüğe çıkar): 88 90 95
```

Python. `heapq` liste üzerinde çalışan fonksiyonlardan oluşur ve yalnızca en küçük öbek kurar (`heapify`, `heappush`, `heappop`; Python Software Foundation, t.y.). En büyük için −x hilesi (`heappush(eb, -x)`, `-heappop(eb)`) ya da `nlargest`; öncelikleri eşit görevlerde ekleme sırasını korumak için (öncelik, sıra, ad) demeti kullanılır (Kavram 3):

ÇIKTI

```
$ python3 H07_K03_heapq.py
heapify      : [45, 52, 63, 88, 84, 90, 71, 95]
heappop x3   : [45, 52, 63]
en büyük x3  : [95, 90, 88]
nlargest(3)  : [95, 90, 88]
görev sırası : ['sinav', 'proje', 'rapor', 'odev']
```

Kararlılık sözleşmesi. C++ `std::sort` kararlılık güvencesi vermez, `std::stable_sort` verir (Kavram 6: `std::stable_sort: sıralı, kararlı mı? evet`; `std::sort : sıralı, kararlı mı? HAYIR (bu derleyicide, bu girdide)`). Python'un `sorted` ve `list.sort`'u kararlıdır; bu yüzden "önce ada, sonra nota göre sırala" iki geçişi (not, ad) anahtarıyla tek sıralamaya eşittir (Kavram 5: `(not, ad) anahtarı aynı mı? evet`). C'nin `qsort`'u için standart algoritma da kararlılık da belirtmez (ISO/IEC, 2018); bir kütüphanede kararlı görünmesi güvence değildir. **Bit işlemleri:** Python tamsayısı sınırsızdır; negatif sayı sağa kaydırıldığında işaret biti sonsuza dek genişler (`-5 >> 24` = −1), bu yüzden taban sıralamasında işaretli anahtarlar önce kaydırılmalıdır (Hata Avcısı 7).

9. Mühendis Gözüyle: Veri Profiline Göre Seçim

- **Önce veriyi tanıyın:** kaç kayıt, anahtar türü ve aralığı, girdi kısmen sıralı mı, eşitlerin sırası önemli mi, bellek kısıtı var mı? Bölüm 4.7'deki karar akışı bu soruların sırasıdır; yanıtları ölçmeden tahmin etmek yaygın bir hatadır.
- **Kütüphane varsayılandır, ama sözleşmesini okuyun:** `qsort` ve `std::sort` kararlılık vermez, `std::stable_sort` ve Python `sorted` verir. Eşit anahtarlı kayıtlar olmadan yazılan testler bu farkı yakalamaz.
- **Doğrusal sıralamanın koşulu bir güvenlik sorunudur:** aralık dışı anahtar C'de sınır dışı yazmadır; bu makinede `double free` ile çöktü, başka yerde sessizce yanlış sonuç verebilir (Hata Avcısı 4). Girdiyi denetleyin, ASan ile sınayın.
- **Bellek ve en kötü durum da birer maliyettir:** sayaç dizisi K ile, birleştirmelinin tamponu n ile büyür; öbek sıralaması O(1) ek bellekle ve en kötü durumda da O(n log n) çalışır (gömülü sistem, gerçek zamanlı sınır, saldırgan girdi).
- **Ölçüm bir deneydir:** kâhinle doğrulayın, işlemci zamanını duvar saatinden ayırın, tekrar edip ortanca alın, kestirimi ölçüm gibi sunmayın, makineyi ve sınırlılıkları yazın (Bölüm 6). Ara projenin ölçüm raporu tam olarak bunu ister.

10. Ara Proje: Mini-OBS 2 v2.6 ve Sıralama Ölçüm Raporu

Ara proje dönemin ikinci modülünü (H4–H7) bütünleştirir. Veriliş: **H7**; teslim: **H8'in ilk dersinin başı**; çalışma biçimi: **bireysel** (kesin tarih, saat ve teslim ortamı derste ve ders sayfasında duyurulur). İki ürün teslim edilir: (1) H4–H7'de eklenen bütün arama ve sıralama yeteneklerini taşıyan, kendi başına derlenen **Mini-OBS 2 v2.6**; (2) v2.4–v2.6'da yazdığınız sıralama yöntemlerini C kütüphanesinin `qsort`'uyla aynı ölçüm düzeneğinde karşılaştıran bir **ölçüm raporu**. Ayrıntılar proje klasöründeki `ARA_PROJE_YONERGESI.md` (yönerge) ve `ARA_PROJE_RAPOR_SABLONU.md` (şablon) dosyalarındadır; bu bölüm onların özetidir.

Önerilen plan (yönerge § 3, H7 → H8, yaklaşık bir hafta):

1. **1.–2. gün:** `obek.c`, `dogrusal.c`; elle izlenen birim testleri; kâhin denemeleri; `--hata-goster lsd` ve `suzme`.
2. **3. gün:** `dagilim`, `ilk_k`, çok yollu `birlestir`; `listele`'ye üç yöntem; `make test`, `make asan`.
3. **4. gün:** `make olcum` — prize takılı, diğer programlar kapalı, tek oturumda.
4. **5.–6. gün:** rapor — önce tabloları ve grafikleri betikle CSV'den üretin, sonra yorumu yazın.
5. **7. gün:** teslim öncesi denetim listesi (yönerge § 11), temiz bir kopyada.

10.1 Kapsam: v2.3–v2.6

Tablo 7.15
Ara projenin kapsadığı sürümler ve öğrenme çıktıları (yönerge § 1; modül ve program listesi § 2'de)

Sürüm	Öğrenme çıktısı (Yol Haritası, bölüm 5)	Projede karşılığı
v2.3 (H4)	ÖÇ 4.2, 4.3 — ikili arama, lower_bound, ekleme konumu	numarayla bulma, sıralı ekleme, soyad önekiyle arama
v2.4 (H5)	ÖÇ 5.2, 5.4, 5.5 — temel sıralamalar, karşılaştırıcı, kararlılık	çok ölçütlü listeler, Türkçe karşılaştırıcı
v2.5 (H6)	ÖÇ 6.2, 6.3, 6.5 — birleştirmeli ve hızlı sıralama, quickselect	toplu yükleme, yinelenen tespiti, şube birleştirme, ortanca

Sürüm	Öğrenme çıktısı (Yol Haritası, bölüm 5)	Projede karşılığı
v2.6 (H7)	ÖÇ 7.2, 7.3, 7.4 — öbek, öncelik kuyruğu, sayma ve taban sıralaması	not dağılımı, ilk-k listesi, numaraya taban sıralaması, k şubenin birleştirilmesi
rapor	**ÖÇ 7.5** — veri özelliklerine göre gerekçeli algoritma seçimi	ölçüm raporu ve seçim rehberi

10.2 v2.6 gereksinimleri ve ipuçları

- `obek.h/.c`: `asagi_suz_say`, `obek_kur_say` (aşağıdan yukarıya), `obek_sirala_say`, `ilk_k_say`; değişmez yorumları; v2.4 arayüzü ve sayaçları.
- `dogrusal.h/.c`: `sayma_sirala_say` (kararlı; artan ve azalan; aralık dışı anahtarda false), `taban_sirala_say` (LSD, taban 256, geçiş sayısı en büyük anahtardan).
- `dagilim` programı: aralık tablosu (genişlik seçilebilir), ortanca, en sık not, `--liste K` (kararlı sayma), `--denetle`.
- `ilk_k` programı: ölçüte göre ilk K (öbek, Θ(n log k)), numarayla tamamlanmış ölçüt, `--denetle`.
- `listele`'ye `obek`, `sayma` (basari/alg/mat/fiz; iki kararlı geçiş), `taban` (no).
- Testler: elle izlenen sayaçlar (öbek kurma), kararlılık (sayma, taban), kâhinler (qsort + kimlik; bütün liste; uç uca + qsort).
- Ölçüm raporu: dokuz algoritma × dört girdi türü × n (karesel üçlü ≥ 8 000'e, diğerleri ≥ 10^6'ya kadar); tablolar, log–log grafikler, eğim ve kuramla karşılaştırma, kararlılık sağlaması, anahtar aralığı deneyi, seçim rehberi, makine bilgisi, sınırlılıklar.

> MİNİ-OBS 2 PROJESİ · **İpuçları (v2.6 belgesinden)**
>
> - Öbekte indisleri kâğıtta çizin: 0 tabanlı dizide çocuklar 2i + 1 ve 2i + 2; son iç düğüm ⌊n/2⌋ − 1.
> - Aşağı süzmede iki çocuğu da karşılaştırın; sağ çocuk yoksa (2i + 2 ≥ m) yalnız sol çocuk.
> - Kurma döngüsünü `size_t` ile geriye doğru yazarken `for (size_t i = n / 2; i-- > 0; )` kalıbını kullanın (i ≥ 0 koşulu işaretsizde hep doğrudur).
> - Sayma sıralamasında önek toplamını "İLK konum" olarak tutun ve girdiyi **baştan sona** dolaşın: kararlılık buradadır. Sondan başa dolaşacaksanız "son konum" tutmalısınız.
> - Taban sıralamasında tampon ile diziyi her geçişte değiştirin; tek sayıda geçişte sonucu diziye geri kopyalamayı unutmayın.
> - İlk-k için öbeğin kökü "seçilenlerin en kötüsü" olmalıdır: başarı listesinde kökte en DÜŞÜK başarılı durur.
> - Kayan nokta değerleri `==` ile eşitlik için karşılaştırmayın; iki ondalıklı veriyi tamsayıya çevirip karşılaştırın.
> - Rapor için önce CSV'yi üretin, tabloları ve grafikleri betikle oluşturun: sayıları elle kopyalamayın.

10.3 Kabul ölçütleri ve test senaryoları

Değerlendirmeye alınma koşulları: temiz bir kopyada derleme hatası olmaması, `make test`'in çalışması, raporun ve sayılarını üreten CSV'lerin teslimde bulunması, derleme ürünü ve büyük veri dosyası olmaması (yönerge § 7.1). Yönerge § 7.2'deki 18 komut teslim klasörünüzde otomatik çalıştırılır; çıkış kodları birebir denetlenir. v2.6'nın kendi kabul ölçütleri:

1. `make` ve `make test` sıfır uyarıyla derlenir; v1.0'ın 99 doğrulaması ve önceki sürümlerin testleri dahil bütün birim testleri ve `test_kahin` (300 deneme) geçer.

2. Öbek kurma 4 1 3 2 16 9 10 14 8 7 → 16 14 10 8 7 9 3 2 4 1; öbek sıralaması (2,0) (2,1) (1,2) dizisini (1,2) (2,1) (2,0) yapar (kararsız).
3. Sayma sıralaması anahtarlar 3 1 2 1 0 3 için kimlikleri 4 1 3 2 0 5 sırasında verir (kararlı); `toplama` n + K = 10.
4. Taban sıralaması en büyük anahtar 255 iken 1, 256 iken 2 geçiş yapar; 10 haneli öğrenci numaralarında 4 geçiş (`olcer kiyas_taban`: "geçiş = 4", "kuram: UYUMLU").
5. `dagilim ornek_veri/ogrenciler.csv --liste 5 --denetle`: ortanca 77,5; liste EMRE ÇINAR, İSMAİL ÖZTÜRK, CAN YILMAZ, AYŞE NUR YILMAZ, ZEYNEP ÇELİK; "Kâhin … AYNI"; çıkış 0.
6. `ilk_k ornek_veri/ogrenciler.csv --k 3 --denetle`: 2301010050, 2301010045, 2301010048; "AYNI"; çıkış 0.
7. `birlestir` A + B + C: 20 kayıt numara sırasında, ilk kayıt 2301010040; A + C + çakışma dosyası: "ÇAKIŞMA: 1 numara … 2301010047", çıkış 1.
8. `listele testler/kararlilik.csv --yontem hepsi`: birlestirmeli ve sayma "EVET / EVET"; obek, hizli ve secmeli "HAYIR / HAYIR".
9. İki kayıtlık eşit ortalama dosyasında (v2.6 belgesi § 4.7) `listele --olcut alg` AK'yi ZEN'den önce verir.
10. Kendi makinenizde 10^6 karışık numarada taban sıralaması birleştirmeli sıralamadan hızlıdır (bu makinede 3,4 kat); n = 10^5'te ilk-10, bütün listeyi sıralamaktan en az 10 kat hızlıdır (bu makinede 61 kat).
11. `--hata-goster lsd` ve `--hata-goster suzme` uyuşmazlığı ve en küçük karşı örneği basar, 1 ile çıkar; `make asan` temizdir.

Sürüm belgesinin § 9'unda ayrıca 12 test senaryosu vardır (öbek kurma, öbek sıralaması, ilk-3, kararlı sayma, aralık dışı anahtar, taban geçiş sayısı, kararsız geçiş gösterimi, kullanım iletileri); kendi testlerinizin en azından bunları kapsaması beklenir.

10.4 Ölçüm raporu

- **Dokuz yöntem × dört girdi türü × n:** seçmeli, kabarcık, araya ekleme, birleştirmeli, hızlı, öbek, sayma, taban ve `qsort`; karışık, sıralı, ters ve çok tekrarlı girdi; karesel üçlü n = 1 000–8 000 (karışıkta 16 000 önerilir), diğerleri n = 10^3–10^6.
- **Düzen:** her nokta için 1 ısınma + en az 5 tekrar, ortanca (10^6'da en az 3 tekrar); `-O2`; sayaçlar süreden ayrı çalıştırmada (Sedgewick & Wayne, 2011).
- **Ek deneyler:** anahtar aralığı (n = 10^5; K = 10 … 10^7), ilk-k (k = 1 … 10^5), çok yollu birleştirme (k = 2 … 64) ve Mini-OBS'ye yansıma (v2.3–v2.5 ölçümlerinizden: numarayla bulma, yükleme, yinelenen tespiti).
- **Kuramla karşılaştırma:** oran testleri, log–log eğimler, karşılaştırma sayısının n $\log_2$ n'ye oranı ve $\log_2$ n! alt sınırı, kapalı biçim sayaçlar (Cormen vd., 2022; Knuth, 1998).
- **Tartışma:** kararlılık sağlaması, önbellek ve sabitlerin etkisi, algoritma seçim rehberi, sınırlılıklar, makine bilgisi.
- **Biçim:** en çok 12 sayfa (ekler hariç), PDF + Markdown kaynağı; her sayı teslim edilen CSV'lerden betikle üretilir.

Rapor şablonunun bölümleri: Özet · 1. Amaç ve araştırma soruları · 2. Yöntem · 3. Sonuçlar · 4. Tartışma · 5. Geçerlilik tehditleri ve sınırlılıklar · 6. Sonuç · Ek A. Yeniden üretme · Ek B. Ham veri dosyaları · Ek C. Beyan · Kaynaklar. Sonuçlar bölümü sekiz alt bölümden oluşur (3.1 Karesel yöntemler; 3.2 Θ(n log n) ve

doğrusal yöntemler; 3.3 Büyüme eğimleri; 3.4 Karşılaştırma ve iş sayıları; 3.5 Kararlılık; 3.6 Anahtar aralığı; 3.7 Öbek uygulamaları; 3.8 Mini-OBS'ye yansıması).

> NOT · **Bu haftanın deneyi ile ara projenin ölçümü aynı şey değildir**
>
> Haftanın Deneyi (Bölüm 6) sekiz algoritmayı (kabarcık yok, kütüphane var) Örnek 12'nin dizileri üzerinde ölçer; ara proje ise **dokuz yöntemi** (seçmeli, kabarcık, araya ekleme, birleştirmeli, hızlı, öbek, sayma, taban, `qsort`) Mini-OBS 2'nin kayıtları ve kendi ölçüm düzeneği üzerinde ölçmenizi ister. Bölüm 6 bir **örnektir**: raporunuzdaki her sayı kendi CSV'lerinizden, kendi makinenizden gelmelidir. Referans rapordaki ya da bu notlardaki sayıları aktarmak veri uydurmadır (yönerge § 9).

10.5 Puanlama

Tablo 7.16
Ara proje puanlaması (yönerge § 8; düzeylerin tanımları yönergededir)

Ölçüt	Puan	Ölçüt içindeki dağılım
İşlevsel doğruluk	25	v2.3 5 · v2.4 5 · v2.5 6 · v2.6 9.
Algoritma seçimi ve analiz	20	kimlik kartları ve karmaşıklık 6 · değişmezler, ön/son koşullar 4 · sayaçların kapalı biçimleri ve kuram denetimleri 4 · Mini-OBS işlerinde seçimin gerekçesi (kararlılık, anahtar aralığı, bellek) 6.
Test ve doğrulama	15	elle izlenmiş birim testleri 4 · kâhin testleri (farklı yol, tohum, sınır durumları) 5 · kasıtlı hata gösterimi ve küçük karşı örnek 3 · otomatik çalıştırma ve regresyon 3.
Ölçüm raporu	25	yöntem ve ortam 4 · tablolar ve grafikler 7 · kuramla karşılaştırma 7 · tartışma ve seçim rehberi 4 · sınırlılıklar ve yeniden üretilebilirlik 3.
Kod kalitesi ve bellek güvenliği	10	uyarısız derleme 2 · ASan/UBSan temiz 3 · modülerlik, adlandırma, sözleşme yorumları, 80 sütun 3 · Makefile hedefleri 2.
Kısa savunma	5	yaklaşık 5 dakika; kendi kodunuzdan bir fonksiyon ve raporunuzdan bir bulgu.

Yönerge § 10'daki 21 sık hatadan bu haftayı doğrudan ilgilendirenler: öbek kurmayı i = 0'dan başlatmak; aşağı süzmede yalnız sol çocuğa bakmak; işaretsiz sayaçla `i >= 0` koşulu (sonsuz döngü); sayma sıralamasında aralık dışı anahtarı denetlememek; yerleştirmeyi sondan başa yaparken "ilk konum" tutmak (kararsız); taban sıralamasında kararsız ara sıralama (`--hata-goster lsd`: [257, 256] sıralanmaz); tek sayıda geçişte sonucu diziye geri kopyalamamak; kayan nokta ortalamalarını eşitlik için doğrudan karşılaştırmak: double'da 0,4 · 3 + 0,6 · 0 ile 0,4 · 0 + 0,6 · 2 farklıdır (v2.6 belgesi § 4.7); ilk-k'da ölçütü numarayla tamamlamamak: eşitlerde sonuç tek değildir ve kâhinle karşılaştırma rastgele başarısız olur. Teslimden önce yönerge § 11'deki denetim listesini temiz bir kopyada uygulayın.

11. Alıştırmalar ve Atölyeler

Alıştırmalar yedi türdedir: A) izleme ve çıktı tahmini, B) karmaşıklık, C) değişmez yazma, D) hata bulma, E) Parsons bulmacası (Parsons & Haden, 2006), F) kod yazma, G) diller arası okuma. Kodu verilen her alıştırmanın dosyası `kodlar/alistirmalar/` altındadır; önce **çalıştırmadan** tahmin edin, sonra çalıştırıp karşılaştırın. Çözümler öğretim üyesindedir.

11.1 İzleme ve çıktı tahmini (A)

Alıştırma 7.1 ★☆☆ *Çıktı tahmini*

Boş bir en büyük öbeğe 30, 50, 20, 70, 40, 90 sırayla ekleniyor (dosyadaki `yukari_suz` Örnek 3'tekiyle aynıdır, 13–22. satırlar). Her eklemeden sonraki diziyi kâğıda yazın; program son diziyi ve toplam takas sayısını ne olarak yazdırır?

alistirmalar/H07_A01_cikti_tahmini.c · satır 26–33

```
26      const int girdi[] = {30, 50, 20, 70, 40, 90};
27      int a[6];
28      int n = 0;
29      for (int k = 0; k < 6; k++) {
30          a[n] = girdi[k];
31          n++;
32          yukari_suz(a, n - 1);
33      }
```

Alıştırma 7.2 ★☆☆ *İzleme*

314 159 265 358 979 323 846 264 sayılarına ondalık LSD taban sıralaması uygulayın ve her geçişten sonraki diziyi yazın (Örnek 10'un 1. bölümündeki biçimde). İkinci geçişte onlar basamağı aynı olan sayı çiftlerini bulun: üçüncü geçişte bunların sırası neden korunmalıdır?

11.2 Karmaşıklık (B)

Alıştırma 7.3 ★★☆ *Karmaşıklık*

Örnek 4'teki `obek_kur_ekleyerek` (`c/H07_04_obek_kur.c · satır 69–74`) artan girdide i. elemanı kaç düzey yukarı taşır? Toplam takası $\Sigma \lfloor \log_2 (i+1) \rfloor$ biçiminde yazıp $\Theta(n \log n)$ olduğunu gösterin. Örnek 4'ün 4. bölümündeki "t/n a" sütununu $\log_2 n$ ile karşılaştırın: oran neden 1'e yaklaşır ama 1'in altında kalır?

Alıştırma 7.4 ★★☆ *Karmaşıklık*

32 bitlik $n = 10^6$ anahtarı r bitlik basamaklarla taban sıralamasıyla sıralamak $\lceil 32/r \rceil \cdot (2n + 2^r - 1)$ adım ister (Örnek 10'un formülü, $b = 2^r$). $r \in \{4, 8, 11, 16\}$ için adım sayısını ve sayaç dizisinin belleğini (sayaç başına 4 bayt) hesaplayın. Hangi r'yi seçerdiniz? Yalnızca adım sayısına bakmak neden yetmez?

11.3 Değişmez yazma (C)

Alıştırma 7.5 ★★☆ *Değişmez yaz*

Öbek sıralamasının ikinci döngüsü (`for (son = n − 1; son > 0; son--)`) için değişmezi a[0..son] ve a[son + 1..n − 1] cinsinden yazın. Takas ve süzmenin değişmezi koruduğunu, döngü bittiğinde dizinin sıralı olduğunu gösterin.

Alıştırma 7.6 ★★☆ *Değişmez yaz*

Örnek 9'daki kararlı sayma sıralamasının yerleştirme döngüsü (sağdan sola) için önek[v]'nin anlamını bir değişmez olarak yazın ve kararlılığı bu değişmezden çıkarın. Aynı önek dizisiyle girdiyi soldan sağa dolaşmak hangi adımda değişmezi bozar (Örnek 9, yol 3)?

11.4 Hata bulma (D)

Alıştırma 7.7 ★★☆ *Hata bul*

Program Örnek 5'in girdisini sıralamalı, ama çıktı sıralı değil. Hatalı satırı bulun. Düzeltmeden önce tahmin edin: hata hangi elemanları yanlış yerde bırakır?

alistirmalar/H07_A02_hata_bul_obek_siralamasi.c · satır 33–42

```
33  static void obek_siralamasi(int *a, int n)
34  {
35      for (int i = n / 2 - 1; i >= 0; i--) {
36          asagi_suz(a, n, i);
37      }
38      for (int son = n - 1; son > 0; son--) {
39          takas(&a[0], &a[son]);
40          asagi_suz(a, son + 1, 0);
41      }
42  }
```

Alıştırma 7.8 ★★☆ *Hata bul*

Kararlı sayma bazı kayıtları kaybediyor, bazı konumları boş bırakıyor. Hatalı satırı bulun. Hangi anahtarın öneki yanlış hesaplanır ve bu hata başka anahtarlı kayıtları da neden kaybettirir?

alistirmalar/H07_A03_hata_bul_onek.c · satır 18–30

```
18  static void kararli_sayma(const Kayit *a, Kayit *b, int n)
19  {
20      int onek[K + 1] = {0};
21      for (int i = 0; i < n; i++) {
22          onek[a[i].anahtar]++;
23      }
24      for (int v = 1; v < K; v++) {
25          onek[v] += onek[v - 1];
26      }
27      for (int i = n - 1; i >= 0; i--) {
28          b[--onek[a[i].anahtar]] = a[i];
29      }
30  }
```

11.5 Parsons (E)

Alıştırma 7.9 ★★☆ *Parsons*

Aşağıdaki satırlar en büyük öbekte aşağı süzme fonksiyonunun karıştırılmış satırlarıdır; iki satır fazladır (dosya bu hâliyle derlenmez). Doğru sırayı ve girintiyi yazın; fazla iki satır neden yanlıştır? Sonuç Örnek 3'ün çıkarma adımlarıyla aynı takasları yapmalıdır.

`alistirmalar/H07_A04_parsons_asagi_suz.c · satır 7–23`

```
        i = c;
    return i;
            break;
static int asagi_suz(int *a, int n, int i)
        if (c + 1 < n && a[c + 1] > a[c]) {
        int c = 2 * i;
    while (2 * i + 1 < n) {
        if (a[c] <= a[i]) {
        takas(&a[i], &a[c]);
            c++;
        }
        int c = 2 * i + 1;
        }
    }
{
    while (2 * i < n) {
}
```

11.6 Kod yazma (F)

Alıştırma 7.10 ★☆☆ *Kod yaz*

$n = 10^6$ kişinin yaşını (0–120) ortak rastgele üreteçle (tohum 42) üreten ve sayma sıralamasıyla on yıllık dilimlerdeki dağılımı, ortancayı ve en sık yaşı yazan bir program yazın. Sıralı diziyi `qsort` sonucuyla karşılaştıran bir kâhin ekleyin. Sayaç dizisi kaç eleman ve kaç bayttır? C++ ve Python sürümlerini de yazın.

Alıştırma 7.11 ★★☆ *Kod yaz*

Her elemanı sıralı dizideki konumundan en çok k uzakta olan (**k-sıralı**) bir diziyi boyutu k + 1 olan **en küçük** öbekle O(n log k) sürede sıralayan bir fonksiyon yazın. $n = 10^5$ ve k = 1, 4, 16, 64 için karşılaştırma sayısını araya eklemeninkiyle (H5, O(nk)) karşılaştırın. Hangi k'dan sonra öbek kazanır?

Alıştırma 7.12 ★★★ *Kod yaz*

Her düğümün d çocuğu olan **d-li öbeği** yazın (çocuklar d·i + 1 … d·i + d, ebeveyn (i − 1)/d) ve öbek sıralamasında kullanın. n = 50 000 rastgele anahtarda d = 2, 3, 4, 8 için yüksekliği, karşılaştırma ve takas sayılarını ölçün. Takas d ile azalırken karşılaştırma neden aynı ölçüde azalmaz, büyük d'de artar? Hangi d'yi seçerdiniz?

11.7 Diller arası okuma (G)

Alıştırma 7.13 ★★☆ *Diller arası okuma*

Python programı ne yazdırır? İlk `print`'teki liste neden sıralı değildir? Aynı ekleme sırasıyla C++ `std::priority_queue<int>` kullanılsaydı ilk iki çıkarılan değer ne olurdu?

alistirmalar/H07_A05_heapq_okuma.py · satır 10–17

```python
    h = []
    for x in [7, 3, 9, 1, 5]:
        heapq.heappush(h, x)
    print(h)
    print(heapq.heappop(h), heapq.heappop(h))
    print(h[0], len(h))
    heapq.heappush(h, -10)
    print(h[0])
```

Alıştırma 7.14 ★★☆ *Diller arası okuma*

C++ programının çıktısı nedir? `std::pair`'ler nasıl karşılaştırılır? Öncelikleri eşit olanları **geliş sırasıyla** (ilk gelen önce) çıkarmak için ne değiştirilmelidir? Python `heapq` ile karşılığını yazın.

alistirmalar/H07_A06_priority_queue_okuma.cpp · satır 15–23

```cpp
    std::priority_queue<std::pair<int, int>> q;  // (öncelik, sıra)
    q.push({2, 0});
    q.push({5, 1});
    q.push({2, 2});
    q.push({5, 3});
    while (!q.empty()) {
        std::cout << q.top().first << '/' << q.top().second << ' ';
        q.pop();
    }
```

11.8 Atölye 1 · Öbek oyunu (sınıf, ≈ 15 dk)

ALGORİTMA ATÖLYESİ · **Öğrenciler düğüm olur: süzmeyi canlandırın**

- On öğrenci ellerinde birer sayı kartıyla (95, 88, 92, 71, 84, 63, 90, 45, 52, 79) dizi sırasında yan yana durur; yere tebeşirle 0 … 9 indisleri yazılır. Her öğrenci ebeveyninin (⌊(i − 1)/2⌋) ve çocuklarının (2i + 1, 2i + 2) kim olduğunu söyler; ardından aynı öğrenciler ağaç biçiminde dizilir (Şekil 7.2).
- **Ekleme:** 97 kartlı yeni öğrenci 10. konuma gelir; yalnızca ebeveyniyle konuşabilir: “benden küçük müsün?” Takas sayısını sınıf tahmin etsin, sonra canlandırın (Örnek 3: 3 takas).
- **Çıkarma:** kökteki öğrenci kartını teslim eder, son konumdaki öğrenci köke yürür, yalnızca iki çocuğuyla konuşur ve **büyüğüyle** yer değiştirir (ilk çocuğu seçerse ne olur? Hata Avcısı 2). **Kurma:** artan sıralı on kartla i = 4'ten 0'a her öğrenci aşağı süzülür; toplam takası Σ yükseklik = n − popcount(n) ile karşılaştırın ve web sayfasındaki öbek görselleştiricisiyle doğrulayın.

11.9 Atölye 2 · Algoritma seçimi: sekiz senaryo kartı (gruplar, ≈ 20 dk)

Her grup iki kart alır; her kart için bir sıralama algoritması seçer ve gerekçesini **ölçülebilir** bir özellikle yazar (anahtar aralığı, n, girdinin düzeni, kararlılık, bellek, en kötü durum). Sonra gerekçeyi Bölüm 4.7'deki tablo ve Bölüm 6'daki ölçümlerle sınar: “bunu nasıl ölçerdik?” Kartlar birbirine benzer görünebilir; ayrıntıyı okuyun.

Tablo 7.17
Atölye 2 senaryo kartları

#	Senaryo
1	Bir nüfus kaydındaki 10^7 kişinin yaşı (0–120 arası tamsayı) artan sıraya konacak.
2	Bellek kısıtlı bir gömülü sistemde her saniye 4 000 sensör ölçümü (32 bit) sıralanacak; ek dizi ayrılamıyor ve süre sınırı kesin (en kötü durum önemli).
3	10^6 öğrenci numarası (10 haneli, işaretsiz tamsayıya sığıyor) sıralanacak.
4	Zaten numaraya göre sıralı 50 000 kayıtlık listenin sonuna her gün birkaç yeni kayıt ekleniyor; liste yeniden sıralanacak.
5	Öğrenciler önce bölüme, bölüm içinde ortalamaya göre listelenecek; aynı ortalamalılar numara sırasında kalmalı (iki geçiş).
6	10^6 başvurudan en yüksek puanlı 10'u duyurulacak; geri kalanların sırası önemli değil.
7	[0, 1) aralığında düzgün dağılmış 10^6 benzetim sonucu (double) sıralanacak.
8	64 şubenin her biri kendi içinde sıralı not listesini gönderdi; tek bir sıralı liste gerekiyor.

11.10 Atölye 3 · Ara proje çalışma oturumu ve kod incelemesi (eşli, ≈ 30 dk)

ALGORİTMA ATÖLYESİ · **Mini-OBS 2 v2.6'ya başlangıç ve eşli kod incelemesi**

İlk 15 dakikada herkes kendi v2.5'i üzerinde `obek.c`'nin `asagi_suz_say` ve `obek_kur_say` fonksiyonlarını ve sayma sıralamasının iskeletini yazar; v2.6 belgesinin test senaryoları 1 (öbek kurma), 4 (kararlı sayma) ve 6 (taban sıralaması) hedeftir. Sonraki 15 dakikada eşler kodlarını değiştirip aşağıdaki listeyle inceler; her madde için kodda satır numarası gösterilir.

- Aşağı süzme iki çocuğu da karşılaştırıyor mu; sağ çocuk yoksa (2i + 2 ≥ m) yalnız sol çocuğa mı bakıyor?
- Kurma döngüsü ⌊n/2⌋ − 1'den mi başlıyor; `size_t` ile geriye sayarken `i >= 0` tuzağına düşülmüş mü?
- Sayma sıralaması aralık dışı anahtarı reddediyor mu; önek dizisi “ilk konum” mu “son konum” mu ve dolaşma yönü onunla uyumlu mu?
- Testler eşit anahtarlı kayıtlarla kararlılığı sınıyor mu; kâhin sınanan fonksiyonu çağırmıyor mu?
- Ölçüm planı: hangi CSV'ler, hangi n'ler, kaç tekrar; makine bilgisi nereye yazılacak (yönerge § 5)?

12. Kendini Değerlendirme ve Vize Hazırlığı

12.1 Bu hafta

Aşağıdakilerin her birini kitaba bakmadan yapabiliyorsanız işaretleyin; yapamadıklarınız için parantez içindeki bölüme dönün.

- ☐ Tam ikili ağacın dizi gösteriminde ebeveyn ve çocuk indislerini 0 tabanlı yazabiliyor, 1 tabanlı formüllerle karıştırmıyorum. (3.1, HA1)
- ☐ Bir öbeğe ekleme ve en büyüğü çıkarmayı hem dizide hem ağaçta adım adım izleyip takas sayısını bulabiliyorum. (4.1)
- ☐ Aşağıdan yukarıya öbek kurmanın neden O(n) olduğunu düzey toplamıyla açıklayabiliyorum. (4.2)

- ☐ Öbek sıralamasını yazabiliyor; yerinde, O(n log n) ve kararsız olduğunu gerekçelendirebiliyorum. (4.3)
- ☐ Öncelik kuyruğunu ilk-k ve k yollu birleştirmeye uygulayabiliyor (O(n log k), O(N log k)); C++ `priority_queue` (en büyük) ile Python `heapq` (en küçük) farkını biliyorum. (5.2, 5.3, 8)
- ☐ Kararlı sayma sıralamasını önek toplamla yazabiliyor ve yerleştirme yönünün kararlılığa etkisini açıklayabiliyorum. (4.4)
- ☐ LSD taban sıralamasının doğruluğunun neden ara geçişlerin kararlılığına dayandığını gösterebiliyorum. (4.5, HA5)
- ☐ Ω(n log n) alt sınırının kapsamını ve doğrusal zamanlı sıralamaların koşullarını açıklayabiliyorum. (3.3)
- ☐ Bir veri profili için sıralama algoritmasını ölçülebilir gerekçeyle seçebiliyorum. (4.7, Atölye 2)

12.2 Vize hazırlığı (H1–H7)

Vize H1–H7'yi kapsar ve sınav C ile yapılır. Soru türleri Yol Haritası Tablo 10.2'dekilerdir: algoritma izleme, karmaşıklık belirleme, değişmez, hata bulma, Parsons ve kod tamamlama, strateji seçimi, sınırlı diller arası okuma ve kod yazma. Web sayfasındaki **vize hazırlık soru bankasında** 47 çoktan seçmeli soru vardır (her haftadan en az beş; hafta ve türe göre süzülebilir; her yanıt açıklamalıdır). Şekil 7.15 haftaların temel kavramlarını ve bankadaki soruların dağılımını gösterir.

Şekil 7.15
Vize konu haritası (H1–H7): temel kavramlar (Yol Haritası Tablo 4.1) ve soru bankasındaki soruların türlere dağılımı

hafta · başlık ve temel kavramlar (Tablo 4.1)		izleme	karmaşıklık	değişmez	hata bulma	Parsons / tamamlama	strateji seçimi	diller arası okuma
H1	Algoritmalara Dönüş: Doğruluk, döngü değişmezi, kâhinle test, deneysel ölçüm, üç dilli çalışma düzeni	1	·	1	2	1	1	1
H2	Algoritma Analizi: T(n), O/Ω/Θ, büyüme sınıfları, oran testi, Eratosthenes kalburu, hızlı üs alma	1	2	1	1	1	·	1
H3	Özyineleme ile Arama Uzayını Dolaşmak: Özyineleme ağacı, alt küme/permütasyon üretimi, geri izleme, budama	1	1	·	1	1	1	1
H4	Arama Algoritmaları: Doğrusal ve ikili arama, [lo, hi) değişmezi, lower/upper_bound, cevap üzerinde ikili arama	1	1	1	1	·	1	1
H5	Temel Sıralama Algoritmaları: Seçmeli, kabarcık ve araya ekleme sıralaması; ters çevrim, kararlılık, qsort karşılaştırıcısı	1	1	1	1	1	1	1
H6	Böl ve Fethet: Birleştirmeli ve hızlı sıralama, bölümleme, ana teorem, Ω(n log n) alt sınırı, quickselect	1	1	1	1	1	1	1
H7	Sıralamanın Ötesi: İkili öbek, öbek sıralaması, öncelik kuyruğu, sayma ve taban sıralaması	1	1	1	1	1	1	1

Hücre: vize hazırlık soru bankasındaki soru sayısı (toplam 47; Tablo 10.2 soru türleri). Kod yazma türü alıştırmalarda.

- **Her hafta için bir "çekirdek" program yazın** (önce kâğıtta, sonra derleyerek): H1 kâhinli test, H2 işlem sayacı, H3 geri izleme şablonu, H4 yarı açık aralıkla ikili arama, H5 araya ekleme, H6 bölümleme ve birleştirme, H7 aşağı süzme ve kararlı sayma.
- **İzlemeyi kâğıtta yapın ve programla karşılaştırın;** karmaşıklığı gerekçesiyle yazın (hangi döngü kaç kez, hangi bağıntı, en kötü durum hangi girdide?); her algoritma için bir değişmez cümlesi kurun ([lo, hi), sıralı önek, üç bölge, "i'den sonraki her düğüm bir öbeğin kökü").
- **Hata kalıplarını tanıyın:** her haftanın Hata Avcısı tablosu sınavın hata bulma sorularının kaynağıdır (taşma, sonsuz döngü, kararsızlık, 0/1 tabanlı indis).
- **Bankayı türe göre çalışın:** en çok yanlış yaptığınız türü süzün; açıklamayı okumadan önce neden yanlış yaptığınızı yazın.

13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · **Bu haftanın konusu nerede derinleşiyor?**

- H8 (dinamik diziler ve bağlı listeler): Kavram 7'deki büyüyen öbek dizisi `realloc` ile büyür; ekleme maliyetinin **amortize** analizi H8'dedir.
- H12 (açgözlü algoritmalar): öncelik kuyruğu Dijkstra'nın en kısa yolunda, Prim'in kapsayan ağacında ve Huffman kodlamasında "en küçüğü al" işlemini yapar; bir anahtarı küçültmek (decrease-key) yerine yinelenen öğeleri eklemek ve eski olanları atlamak (tembel silme) orada işlenir.
- d-ary, binom ve Fibonacci öbekleri gibi yapılar Veri Yapıları ve Algoritma Analizi derslerinde incelenir.
- Taban sıralaması ve sonek dizileri, dizgi algoritmalarındaki ileri uygulamalara açılır.

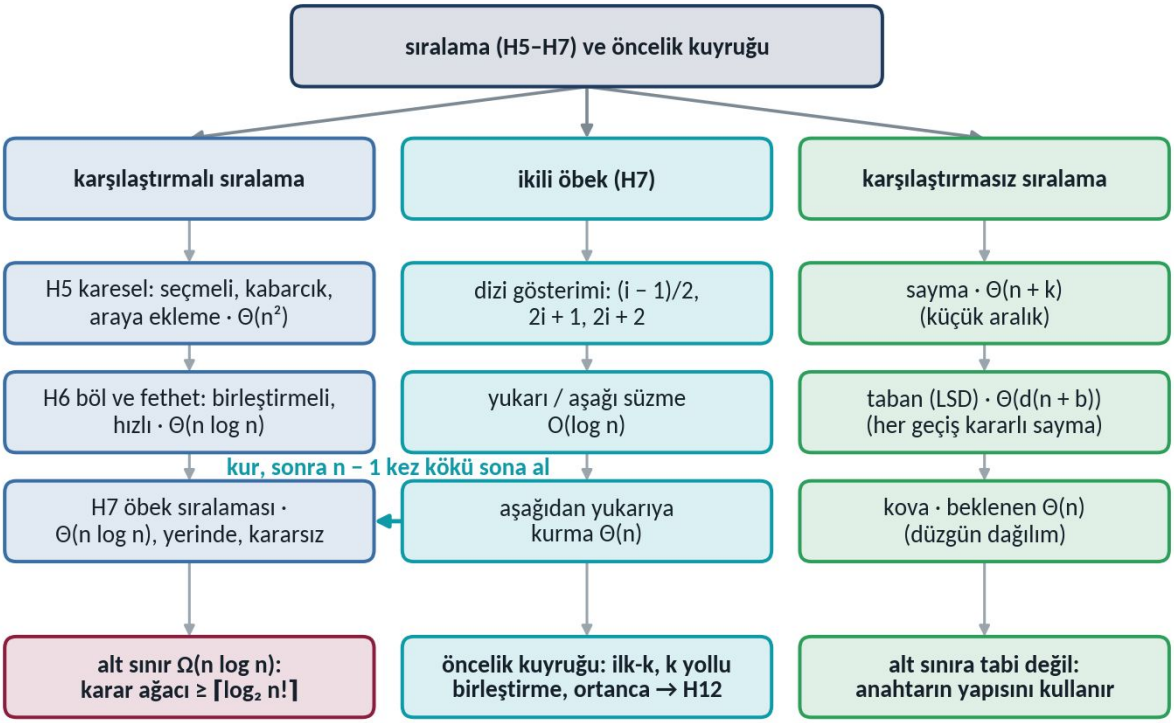
MERAK KUTUSU · **Akan veride ortanca: iki öbek (Örnek 13)**

Sayılar tek tek geliyor; her gelişten sonra o ana kadarki ortanca gerekiyor. Her seferinde sıralamak O(n log n), sıralı bir diziye araya eklemek O(n) ister. **İki öbek** yeter: küçük yarı bir **en büyük öbekte** (alt), büyük yarı bir **en küçük öbekte** (üst) tutulur. Yeni sayı alt kökünden küçük-eşitse alta, değilse üste eklenir; boyutlar arasındaki fark 1'i aşarsa bir kök öbür öbeğe taşınır. Ortanca köklerden okunur: O(log n) / sayı. Örnek 13'ün on beş sayılık akışında son durum: `15. gelen 88 ortanca = 38.0 alt (en büyük) [38 34 26 25 10 4 3 14] üst (en küçük) [56 66 82 69 90 94 88]`.

Örnek 13 100 000 sayılık akışta denetim noktalarında sonucu sıralamalı kâhinle karşılaştırır: n = 100 000'de sayı başına 23,34 karşılaştırma ($\log_2 n = 16{,}61$), kâhin "evet". Sıralı diziye araya eklemek n = 2 000'de 987 994 kaydırma, iki öbek 29 898 karşılaştırma ister. Soru: %90'lık dilim için boyut kuralını nasıl değiştirirsiniz?

14. Özet, Kavram Haritası ve Terim Sözlüğü

Şekil 7.16
Hafta 7 kavram haritası



- Tam ikili ağaç bir dizide işaretçisiz tutulur: sol $2i + 1$, sağ $2i + 2$, ebeveyn $\lfloor (i - 1)/2 \rfloor$; yükseklik $\lfloor \log_2 n \rfloor$.
- Öbek özelliği dikey bir sıradır (ebeveyn ≥ çocuk); öbek sıralı değildir, ama kökü en büyüktür. Ekleme yukarı, çıkarma aşağı süzer: O(log n).
- Aşağıdan yukarıya öbek kurma Θ(n)'dir: takas ≤ Σ yükseklik = n − popcount(n) < n. Tek tek eklemek Θ(n log n)'dir.
- Öbek sıralaması yerinde, en kötü durumda Θ(n log n) ve kararsızdır; bu makinede hızlı sıralamanın ≈ 2 katı sürdü (önbellek). Introsort'un yedeğidir.
- Öncelik kuyruğu bir ADT'dir; öbekle ilk-k O(n log k), k yollu birleştirme O(N log k) olur. C++ `priority_queue` en büyük, Python `heapq` en küçük öbektir.
- Karşılaştırmalı sıralama Ω(n log n)'dir (karar ağacı). Sayma, taban ve kova bu modelin dışındadır ve koşulları vardır: küçük aralık, sabit uzunluk, bilinen dağılım.
- Sayma Θ(n + k), taban Θ(d(n + b)) ve ikisi de kararlıdır; taban sıralaması ara geçişlerin kararlılığına dayanır. Bu makinede $n = 10^6$'da sayma 18,7 ms, taban 20,3 ms, `qsort` 142,0 ms.
- Algoritma, verinin profiline göre seçilir ve seçim ölçülerek doğrulanır; ara projenin ölçüm raporu bunun yazılı biçimidir.

Tablo 7.18
Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Not
tam ikili ağaç; dizi gösterimi	complete binary tree; array representation	0 tabanlı formüller

Türkçe	İngilizce	Not
öbek; en büyük / en küçük öbek; öbek özelliği	heap; max-heap / min-heap; heap property	öbek bellek bölgesiyle karıştırmayın
yukarı süzme; aşağı süzme	sift-up; sift-down (heapify)	
öbek kurma; öbek sıralaması	build-heap; heapsort	Williams (1964)
öncelik kuyruğu	priority queue	soyut veri tipi (ADT)
ilk-k; k yollu birleştirme	top-k; k-way merge	
karar ağacı; alt sınır	decision tree; lower bound	"alt sınır" yalnızca Ω anlamında
karşılaştırmasız sıralama; sayma sıralaması; önek toplam	non-comparison sort; counting sort; prefix sum	
taban sıralaması (LSD / MSD); kova sıralaması	radix sort (least / most significant digit); bucket sort	
kararlı; yerinde; uyarlanabilir	stable; in-place; adaptive	Yol Haritası 6.8
yığın; kuyruk	stack; queue	"yığın" her zaman stack

15. Kaynaklar

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Floyd, R. W. (1964). Algorithm 245: Treesort 3. *Communications of the ACM*, *7*(12), 701. https://doi.org/10.1145/355588.365103

ISO/IEC. (2017). *Programming languages — C++* (ISO/IEC Standard No. 14882:2017). International Organization for Standardization.

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2nd ed.). Addison-Wesley.

Musser, D. R. (1997). Introspective sorting and selection algorithms. *Software: Practice and Experience*, *27*(8), 983–993.

OEIS Foundation Inc. (t.y.). *Number of (binary) heaps on n elements* (A056971). The On-Line Encyclopedia of Integer Sequences. Erişim tarihi 13 Ekim 2026, https://oeis.org/A056971

Parsons, D., & Haden, P. (2006). Parson's programming puzzles: A fun and effective learning tool for first programming courses. In *Proceedings of the 8th Australasian Conference on Computing Education* (Vol. 52, pp. 157–163). Australian Computer Society.

Python Software Foundation. (t.y.). *The Python standard library*. Erişim tarihi 12 Ekim 2026, https://docs.python.org/3/library/

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

U.S. Census Bureau. (2024, August 14). *The Hollerith machine*. Erişim tarihi 13 Ekim 2026, https://www.census.gov/about/history/bureau-history/census-innovations/technology/hollerith-machine.html

Williams, J. W. J. (1964). Algorithm 232: Heapsort. *Communications of the ACM*, *7*(6), 347–348. https://doi.org/10.1145/512274.3734138

Ek A. Dil Karşılıkları

Bu ekte örnek merdiveninin on üç basamağının C++ ve Python sürümlerinden çekirdek fonksiyonlar, dosyadaki satır numaralarıyla verilir; tam dosyalar `kodlar/cpp/` ve `kodlar/python/` altındadır. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`); fonksiyon dışındaki farklar not kutusunda açıklanır. Fonksiyon ve değişken adları üç dilde aynıdır. Bölüm 4'te üç dili gösterilen fonksiyonlar burada tekrarlanmaz; aynı dosyaların öteki çekirdek fonksiyonları verilir. Bütün örneklerin çıktısı üç dilde aynıdır (=).

Tablo 7.19
Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H07_05_obek_siralamasi)

Dil	Derleme / çalıştırma	Not
C	`gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c ortak_rastgele.c -o X -lm · ./X`	Örnek 1 ve 2'de ortak_rastgele.c gerekmez
C++	`g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X · ./X`	ortak başlıklar (.hpp) yalnızca #include
Python	`python3 X.py (Windows: py X.py)`	Python 3.10 ve üstü; yalnızca standart kütüphane
Hepsi	`./H07_04_obek_kur [n_ust] · ./H07_05_obek_siralamasi [n_ust] · ./H07_12_siralama_karsilastirma olc [n] [tekrar]`	isteğe bağlı; hatalı argümanda çıkış kodu 2

A.1 Örnek 1 · Dizi ↔ ağaç

C++ **cpp/H07_01_dizi_agac.cpp · satır 23–31**

```cpp
static int duzey(int i)
{
    int d = 0;
    while (i > 0) {
        i = ebeveyn(i);
        d++;
    }
    return d;
}
```

Python **python/H07_01_dizi_agac.py · satır 25–31**

```python
def duzey(i):
    """i. düğümün düzeyi (kök 0): i + 1'in ikilik basamak sayısı - 1"""
    d = 0
    while i > 0:
        i = ebeveyn(i)
        d += 1
    return d
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de `(0 - 1) / 2` sıfıra doğru kesildiği için 0, Python'da `(0 - 1) // 2` aşağı yuvarlandığı için −1 verir: kökün "ebeveyni" iki dilde farklıdır; bu yüzden `ebeveyn(i)` yalnızca `i > 0` iken çağrılır. Satır tamponu C'de `char` dizisi + `memcpy`, C++'ta `std::string::replace`, Python'da karakter listesi + dilim atamasıyla kurulur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 98, 101, 128; Python 14, 69, 86, 107.)

A.2 Örnek 2 · Öbek özelliği denetimi

C++ **cpp/H07_02_obek_mi.cpp · satır 22–31**

```cpp
static int ilk_ihlal(const std::vector<int>& a, int n)
{
    for (int i = 1; i < n; i++) {
        karsilastirma_sayisi++;
        if (a[ebeveyn(i)] < a[i]) {
            return i;
        }
    }
    return -1;
}
```

Python **python/H07_02_obek_mi.py · satır 17–24**

```python
def ilk_ihlal(a, n):
    """İlk ihlal: a[ebeveyn(i)] < a[i] olan en küçük i; ihlal yoksa -1"""
    global karsilastirma_sayisi
    for i in range(1, n):
        karsilastirma_sayisi += 1
        if a[ebeveyn(i)] < a[i]:
            return i
    return -1
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C ve C++'taki `do { … } while (sonraki_permutasyon(p, n));` döngüsünün Python'da doğrudan karşılığı yoktur: `while True:` ve sonda `if not …: break` kullanılır. Takas C'de yardımcı fonksiyonla, C++'ta `std::swap`, Python'da çoklu atamayla yapılır; C++'ta `std::is_heap` ve `std::next_permutation` aynı işleri yapar (kütüphane karşılığı). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 51, 64; Python 14, 50, 95, 102.)

A.3 Örnek 3 · Ekleme ve en büyüğü çıkarma

C++ **cpp/H07_03_ekle_cikar.cpp · satır 37–61**

```cpp
static int yukari_suz(std::vector<int>& a, int i)
{
    while (i > 0 && buyuk_mu(a[i], a[ebeveyn(i)])) {
        takas(a[i], a[ebeveyn(i)]);
        i = ebeveyn(i);
    }
    return i;
```

```cpp
}

// a[i], büyük çocuğundan küçük oldukça aşağı taşınır (a[0..n-1])
static int asagi_suz(std::vector<int>& a, int n, int i)
{
    while (2 * i + 1 < n) {
        int c = 2 * i + 1;  // sol çocuk
        if (c + 1 < n && buyuk_mu(a[c + 1], a[c])) {
            c++;  // sağ çocuk daha büyük
        }
        if (!buyuk_mu(a[c], a[i])) {
            break;  // öbek özelliği sağlandı
        }
        takas(a[i], a[c]);
        i = c;
    }
    return i;
}
```

Python **python/H07_03_ekle_cikar.py · satır 33–51**

```python
def yukari_suz(a, i):
    """a[i] ebeveyninden büyük oldukça yukarı taşınır; son konumu döndürür"""
    while i > 0 and buyuk_mu(a[i], a[ebeveyn(i)]):
        takas(a, i, ebeveyn(i))
        i = ebeveyn(i)
    return i


def asagi_suz(a, n, i):
    """a[i], büyük çocuğundan küçük oldukça aşağı taşınır (a[0..n-1])"""
    while 2 * i + 1 < n:
        c = 2 * i + 1  # sol çocuk
        if c + 1 < n and buyuk_mu(a[c + 1], a[c]):
            c += 1  # sağ çocuk daha büyük
        if not buyuk_mu(a[c], a[i]):
            break  # öbek özelliği sağlandı
        takas(a, i, c)
        i = c
    return i
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de öbek sabit kapasiteli bir dizi ve ayrı tutulan boyuttur (`obege_ekle(a, &n, x)`); C++'ta `std::vector` boyutunu `push_back`/`pop_back` ile, Python'da liste `append`/`pop` ile kendisi yönetir. Dizilerin eşitliği C'de döngüyle, C++ ve Python'da `==` ile denetlenir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 66, 74, 75, 138; Python 56, 63, 64, 112.)

A.4 Örnek 4 · Aşağıdan yukarıya öbek kurma

C++ cpp/H07_04_obek_kur.cpp · satır 90–107

```cpp
static int yukseklik(int i, int n)
{
    int h = 0;
    while (2 * i + 1 < n) {
        i = 2 * i + 1;
        h++;
    }
    return h;
}

static long long yukseklik_toplami(int n)
{
    long long t = 0;
    for (int i = 0; i < n; i++) {
        t += yukseklik(i, n);
    }
    return t;
}
```

Python python/H07_04_obek_kur.py · satır 71–84

```python
def yukseklik(i, n):
    """i. düğümün yüksekliği: altındaki en uzun yol (en soldaki yol)"""
    h = 0
    while 2 * i + 1 < n:
        i = 2 * i + 1
        h += 1
    return h


def yukseklik_toplami(n):
    t = 0
    for i in range(n):
        t += yukseklik(i, n)
    return t
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> C'deki `for (int i = n / 2 - 1; i >= 0; i--)` Python'da `range(n // 2 - 1, -1, -1)` olur: bitiş değeri −1 dışlayıcıdır, böylece i = 0 da işlenir. Komut satırı sayısı C'de `strtol` + son karakter denetimi, C++'ta `std::stol` + `try`, Python'da `int()` + `ValueError` ile okunur; hatalı kullanımda üç dilde çıkış kodu 2'dir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 257, 258; Python 54, 204, 205.)

A.5 Örnek 5 · Öbek sıralaması

C++ cpp/H07_05_obek_siralamasi.cpp · satır 90–100

```cpp
static void kayit_siralamasi(std::vector<Kayit>& a)
{
    int n = static_cast<int>(a.size());
    for (int i = n / 2 - 1; i >= 0; i--) {
        kayit_suz(a, n, i);
    }
    for (int son = n - 1; son > 0; son--) {
        std::swap(a[0], a[son]);
        kayit_suz(a, son, 0);
    }
}
```

Python python/H07_05_obek_siralamasi.py · satır 66–72

```python
def kayit_siralamasi(a):
    n = len(a)
    for i in range(n // 2 - 1, -1, -1):
        kayit_suz(a, n, i)
    for son in range(n - 1, 0, -1):
        a[0], a[son] = a[son], a[0]
        kayit_suz(a, son, 0)
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> Kâhin C'de `qsort` + karşılaştırma fonksiyonu, C++'ta `std::sort`, Python'da `sorted`'dır; sıralamanın kendisi üç dilde elle yazılmıştır. Kayıt C'de `struct` + `const char *`, C++'ta `struct` + `std::string`, Python'da `(not, ad)` demetidir. Alan adı `not_`: `not` C++'ta ve Python'da ayrılmış sözcüktür. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 70, 72, 144; Python 49, 62, 108.)

A.6 Örnek 6 · Öncelik kuyruğuyla ilk-k

C++ cpp/H07_06_ilk_k.cpp · satır 61–95

```cpp
static std::vector<Ogrenci> ilk_k(const std::vector<Ogrenci>& a, int k,
                                  bool izle)
{
    std::vector<Ogrenci> h;
    h.reserve(k);
    for (const Ogrenci& x : a) {
        std::string ne;
        if (static_cast<int>(h.size()) < k) {
            h.push_back(x);
            yukari_suz(h, static_cast<int>(h.size()) - 1);
            ne = "eklendi";
        } else if (kotu_mu(h[0], x)) {
            h[0] = x;                       // en kötüyü at
            asagi_suz(h, k, 0);
            degisim_sayisi++;
            ne = "köke yazıldı";
        } else {
```

```
            ne = "atlandı";
        }
        if (izle) {
            std::cout << "  " << x.no << " (" << std::setw(3) << x.not_
                      << ")  kök = " << std::setw(3) << h[0].not_
                      << "  öbek:";
            for (const Ogrenci& y : h) {
                std::cout << " " << y.not_;
            }
            std::cout << "  <- " << ne << "\n";
        }
    }
    for (int son = static_cast<int>(h.size()) - 1; son > 0; son--) {
        std::swap(h[0], h[son]);                    // en kötü sona
        asagi_suz(h, son, 0);
    }
    return h;
}
```

Python **python/H07_06_ilk_k.py · satır 42–65**

```
def ilk_k(a, k, izle):
    """Her kayıt için: öbek dolmadıysa ekle; doluysa ve kökten iyiyse kökü
    değiştir. Sonunda öbek yerinde sıralanır: h[0] en iyi."""
    global degisim_sayisi
    h = []
    for x in a:
        if len(h) < k:
            h.append(x)
            yukari_suz(h, len(h) - 1)
            ne = "eklendi"
        elif kotu_mu(h[0], x):
            h[0] = x                          # en kötüyü at
            asagi_suz(h, k, 0)
            degisim_sayisi += 1
            ne = "köke yazıldı"
        else:
            ne = "atlandı"
        if izle:
            print(f"  {x[0]} ({x[1]:3d})  kök = {h[0][1]:3d}  öbek:"
                  + "".join(f" {y[1]}" for y in h) + f"  <- {ne}")
    for son in range(len(h) - 1, 0, -1):            # en kötü sona
        h[0], h[son] = h[son], h[0]
        asagi_suz(h, son, 0)
    return h
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Kâhinin sıralama ölçütü üç dilde farklı yazılır: C'de `qsort` için negatif/sıfır/pozitif döndüren fonksiyon, C++'ta `std::sort` için "önce mi gelir?" sorusunu yanıtlayan `bool` fonksiyon, Python'da `key=lambda r: (-r[1], r[0])` (not azalan, numara artan). Kayıt C/C++'ta `struct`, Python'da `(no, not)` demetidir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 100, 133; Python 21, 89.)

A.7 Örnek 7 · k sıralı listenin birleştirilmesi

C++ cpp/H07_07_k_yollu_birlestirme.cpp · satır 70–105

```cpp
static std::vector<int> k_yollu_birlestir(const Listeler& liste, bool izle)
{
    int k = static_cast<int>(liste.size());
    std::vector<Oge> h;
    std::vector<int> cikti;
    for (int j = 0; j < k; j++) {
        if (!liste[j].empty()) {
            h.push_back({liste[j][0], j, 0});
            yukari_suz(h, static_cast<int>(h.size()) - 1);
        }
    }
    if (izle) {
        std::cout << "  başlangıç öbeği:";
        obek_yaz(h);
    }
    while (!h.empty()) {
        Oge en = h[0];
        cikti.push_back(en.deger);
        const std::vector<int>& lis = liste[en.liste];
        if (en.konum + 1 < static_cast<int>(lis.size())) {
            int s = en.konum + 1;
            h[0] = {lis[s], en.liste, s};
        } else {
            h[0] = h.back();               // liste bitti
            h.pop_back();
        }
        asagi_suz(h, static_cast<int>(h.size()), 0);
        if (izle) {
            std::cout << "  " << std::setw(2) << cikti.size() << ". çıktı "
                      << en.deger << "/" << static_cast<char>('A' + en.liste)
                      << "  öbek:";
            obek_yaz(h);
        }
    }
    return cikti;
}
```

Python python/H07_07_k_yollu_birlestirme.py · satır 46–73

```python
def k_yollu_birlestir(liste, izle):
    """Her listenin ilk elemanı öbeğe; sonra kökü çıktıya yaz ve yerine aynı
    listenin sonraki elemanını koy (liste bittiyse son öğeyi köke taşı)"""
    h = []
    cikti = []
    for j, lis in enumerate(liste):
        if lis:
            h.append((lis[0], j, 0))
            yukari_suz(h, len(h) - 1)
    if izle:
        print("  başlangıç öbeği:", end="")
        obek_yaz(h)
    while h:
```

```
        en = h[0]
        cikti.append(en[0])
        lis = liste[en[1]]
        if en[2] + 1 < len(lis):
            s = en[2] + 1
            h[0] = (lis[s], en[1], s)
        else:
            h[0] = h[-1]                    # liste bitti
            h.pop()
        asagi_suz(h, len(h), 0)
        if izle:
            print(f"  {len(cikti):2d}. çıktı {en[0]}/{chr(ord('A') + en[1])}"
                  "  öbek:", end="")
            obek_yaz(h)
    return cikti
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de listeler işaretçi dizisi + ayrı uzunluk dizisiyle geçirilir ve boş liste `NULL` + 0 uzunlukla gösterilir; C++'ta `std::vector<std::vector<int>>`, Python'da liste listesi kendi boyunu bilir. Öbek öğesi C'de bileşik değişmezle `(Oge){…}`), C++'ta küme başlatıcıyla `{…}`), Python'da demetle kurulur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 66.)

A.8 Örnek 8 · Sayma sıralaması

C++ cpp/H07_08_sayma_siralamasi.cpp · satır 47–61

```
static void maliyet_satiri(int n, int k)
{
    std::vector<int> a = rastgele_notlar(n, k);
    std::vector<int> b = a;
    std::sort(b.begin(), b.end());                      // kâhin
    adim_sayisi = 0;
    sayma_siralamasi(a, k);
    bool dogru = (a == b);
    std::cout << std::fixed << std::setprecision(3) << std::setw(9) << n
              << " " << std::setw(9) << k << " " << std::setw(12)
              << adim_sayisi << " " << std::setw(12) << 2LL * n + k + 1
              << " " << std::setw(9) << static_cast<double>(adim_sayisi) / n
              << " " << std::setw(10) << 4LL * (k + 1) << "  "
              << (dogru ? "evet" : "HAYIR") << "\n";
}
```

Python **python/H07_08_sayma_siralamasi.py · satır 36–45**

```python
def maliyet_satiri(n, k):
    global adim_sayisi
    a = rastgele_notlar(n, k)
    b = sorted(a)                                   # kâhin
    adim_sayisi = 0
    sayma_siralamasi(a, k)
    dogru = (a == b)
    print(f"{n:9d} {k:9d} {adim_sayisi:12d} {2 * n + k + 1:12d} "
          f"{adim_sayisi / n:9.3f} {4 * (k + 1):10d}  "
          + ("evet" if dogru else "HAYIR"))
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> Sayaç dizisi C'de `calloc` ile sıfırlanmış bellekte (sonunda `free`), C++'ta `std::vector<int>(k + 1, 0)`, Python'da `[0] * (k + 1)` ile kurulur. Python listesinin her yuvası bir nesne başvurusudur (64 bitte 8 bayt): C'deki 4 baytlık `int` sayaç dizisinin en az iki katıdır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 25, 44; Python 17, 33, 76.)

A.9 Örnek 9 · Kararlı sayma sıralaması

C++ **cpp/H07_09_kararli_sayma.cpp · satır 32–60**

```cpp
static std::vector<Kayit> kararli_sayma(const std::vector<Kayit>& a, int k,
                                        bool izle)
{
    int n = static_cast<int>(a.size());
    std::vector<Kayit> b(n);
    std::vector<int> onek(k + 1, 0);
    for (const Kayit& r : a) {
        onek[r.not_]++;                     // sıklık
    }
    if (izle) {
        dizi_yaz("sayaç    ", onek);
    }
    for (int v = 1; v <= k; v++) {
        onek[v] += onek[v - 1];             // önek toplam
    }
    if (izle) {
        dizi_yaz("önek top.", onek);
    }
    for (int i = n - 1; i >= 0; i--) {      // sağdan sola: kararlı
        int v = a[i].not_;
        onek[v]--;
        b[onek[v]] = a[i];
        if (izle) {
            std::cout << "  a[" << i << "] = (" << a[i].no << ", " << v
                      << ") -> b[" << onek[v] << "]\n";
        }
    }
    return b;
}
```

Python **python/H07_09_kararli_sayma.py · satır 16–36**

```python
def kararli_sayma(a, k, izle):
    """Yol 1: önek[v] = anahtarı <= v olan kayıt sayısı; sağdan sola
    yerleştir. Kayıt: (no, not) demeti; anahtar not (0..k)"""
    n = len(a)
    b = [None] * n
    onek = [0] * (k + 1)
    for r in a:
        onek[r[1]] += 1                  # sıklık
    if izle:
        dizi_yaz("sayaç    ", onek)
    for v in range(1, k + 1):
        onek[v] += onek[v - 1]           # önek toplam
    if izle:
        dizi_yaz("önek top.", onek)
    for i in range(n - 1, -1, -1):       # sağdan sola: kararlı
        v = a[i][1]
        onek[v] -= 1
        b[onek[v]] = a[i]
        if izle:
            print(f"  a[{i}] = ({a[i][0]}, {v}) -> b[{onek[v]}]")
    return b
```

DİL PENCERESİ · **Farklılaşan satırlar**

C ve C++'taki `b[bas[v]++] = r` tek deyimde hem yazar hem artırır; Python'da `++` yoktur, iki deyim gerekir. Çıktı dizisi C'de çağıranın verdiği dizidir, C++'ta `std::vector<Kayit>` olarak döndürülür, Python'da `[None] * n` ile ayrılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 75; Python 48, 49.)

A.10 Örnek 10 · Taban sıralaması (LSD)

C++ **cpp/H07_10_taban_siralamasi.cpp · satır 67–85**

```cpp
static void taban_siralamasi(std::vector<std::uint32_t>& a, int bit,
                             bool izle)
{
    std::uint32_t maske = (1u << bit) - 1;
    std::vector<std::uint32_t> b(a.size());
    std::vector<int> onek(maske + 1);
    for (int kaydir = 0; kaydir < 32; kaydir += bit) {
        gecis(a, b, kaydir, maske, onek);
        std::swap(a, b);               // rolleri değiştir (O(1))
        if (izle) {
            std::cout << "  bayt " << kaydir / 8 << "   :" << std::hex
                      << std::uppercase << std::setfill('0');
            for (std::uint32_t x : a) {
                std::cout << " " << std::setw(8) << x;
            }
            std::cout << std::dec << std::setfill(' ') << "\n";
        }
    }
}
```

Python **python/H07_10_taban_siralamasi.py · satır 49–61**

```python
def taban_siralamasi(a, bit, izle):
    """bit: basamak genişliği (4, 8 ya da 16); 32 / bit geçiş (çift sayı:
    sonuç yeniden a'da). izle: her geçişten sonra yazdır"""
    maske = (1 << bit) - 1
    b = [0] * len(a)
    onek = [0] * (maske + 1)
    kaynak, hedef = a, b
    for kaydir in range(0, 32, bit):
        gecis(kaynak, hedef, kaydir, maske, onek)
        kaynak, hedef = hedef, kaynak      # rolleri değiştir
        if izle:
            print(f"  bayt {kaydir // 8}   :"
                  + "".join(f" {x:08X}" for x in kaynak))
```

DİL PENCERESİ · **Farklılaşan satırlar**

Bayt C/C++'ta işaretsiz `uint32_t` üzerinde `(x >> kaydir) & maske` ile alınır; Python tamsayısı sınırsızdır ve negatif sayıyı sağa kaydırınca işaret biti sonsuza dek genişler — işaretli sayılar ayrıca ele alınmalıdır (Hata Avcısı 7). C'de tek ifadedeki iki `rastgele_sayi()` çağrısının sırası belirtilmemiştir; bu yüzden iki ayrı deyim yazılır (Python soldan sağa değerlendirir). On altılık yazım: `%08X`, `std::hex + std::setw(8) + std::setfill('0')`, `f"{x:08X}"`. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 90; Python 66.)

A.11 Örnek 11 · Kova sıralaması

C++ **cpp/H07_11_kova_siralamasi.cpp · satır 24–45**

```cpp
static void araya_ekleme(std::vector<double>& a, int bas, int son)
{
    for (int i = bas + 1; i < son; i++) {
        double x = a[i];
        int j = i - 1;
        while (j >= bas) {
            karsilastirma_sayisi++;
            if (a[j] <= x) {
                break;
            }
            a[j + 1] = a[j];               // kaydır
            j--;
        }
        a[j + 1] = x;
    }
}

static int kova_no(double x, int n)
{
    int k = static_cast<int>(x * n);       // 0 <= x < 1
    return k < n ? k : n - 1;
}
```

Python **python/H07_11_kova_siralamasi.py · satır 16–33**

```python
def araya_ekleme(a, bas, son):
    """a[bas..son-1] dilimini araya eklemeyle sıralar"""
    global karsilastirma_sayisi
    for i in range(bas + 1, son):
        x = a[i]
        j = i - 1
        while j >= bas:
            karsilastirma_sayisi += 1
            if a[j] <= x:
                break
            a[j + 1] = a[j]                # kaydır
            j -= 1
        a[j + 1] = x


def kova_no(x, n):
    k = int(x * n)                         # 0 <= x < 1
    return k if k < n else n - 1
```

DİL PENCERESİ · **Farklılaşan satırlar**

Kova numarası C'de `(int)(x * n)`, Python'da `int(x * n)` ile bulunur; ikisi de pozitif sayıyı sıfıra doğru keser. Python'da `a[:] = b` çağıranın listesini yerinde değiştirir; `a = b` yalnızca yerel adı yeniden bağlardı. Ondalıklar üç dilde sabit basamakla (`%.4f`, `std::setprecision(4)`, `:.4f`) yazıldığı için çıktı bayt bayt aynıdır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 60, 82; Python 47, 63.)

A.12 Örnek 12 · Sıralama karşılaştırma deneyi

C++ **cpp/H07_12_siralama_karsilastirma.cpp · satır 194–209**

```cpp
static void sayma_siralamasi(Dizi& a, int k)
{
    int n = static_cast<int>(a.size());
    std::vector<int> sayac(k + 1, 0);
    for (int x : a) {
        sayac[x]++;
    }
    int j = 0;
    for (int v = 0; v <= k; v++) {
        for (int c = 0; c < sayac[v]; c++) {
            a[j++] = v;
        }
    }
    sayac_islemi += n + k + 1;
    yazma_sayisi += n;
}
```

Python **python/H07_12_siralama_karsilastirma.py · satır 159–172**

```python
def sayma_siralamasi(a, k):
    """6. Sayma (Örnek 8): anahtarlar 0..k"""
    global sayac_islemi, yazma_sayisi
    n = len(a)
    sayac = [0] * (k + 1)
    for x in a:
        sayac[x] += 1
    j = 0
    for v in range(k + 1):
        for _ in range(sayac[v]):
            a[j] = v
            j += 1
    sayac_islemi += n + k + 1
    yazma_sayisi += n
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> Hoare bölümlemesindeki `do { i++; } while (…)` Python'da `i += 1` ve ardından `while …: i += 1` olarak yazılır. Kütüphane sıralaması C'de `qsort` (fonksiyon işaretçili karşılaştırma; standart algoritmayı belirtmez), C++'ta `std::sort` (introsort), Python'da `list.sort` (Timsort); üçü de yalnızca kâhin ve süre karşılaştırmasıdır, sayaç içermez. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 32, 228, 237; Python 80, 110, 198.)

A.13 Örnek 13 · Akan veride ortanca

C++ **cpp/H07_13_akan_ortanca.cpp · satır 68–88**

```cpp
static void ortancaya_ekle(Obek& alt, Obek& ust, int x)
{
    if (alt.a.empty() || !ustte_mi(alt, x, alt.a[0])) {
        ekle(alt, x);  // x <= alt'ın tepesi
    } else {
        ekle(ust, x);
    }
    if (alt.a.size() > ust.a.size() + 1) {  // dengele
        ekle(ust, cikar(alt));
    } else if (ust.a.size() > alt.a.size()) {
        ekle(alt, cikar(ust));
    }
}

static double ortanca(const Obek& alt, const Obek& ust)
{
    if (alt.a.size() > ust.a.size()) {
        return alt.a[0];
    }
    return (alt.a[0] + ust.a[0]) / 2.0;
}
```

Python **python/H07_13_akan_ortanca.py · satır 55–71**

```python
def ortancaya_ekle(alt, ust, x):
    """Değişmez: alt'taki her değer <= ust'teki her değer ve
    len(alt.a) == len(ust.a) ya da len(alt.a) == len(ust.a) + 1"""
    if not alt.a or not ustte_mi(alt, x, alt.a[0]):
        ekle(alt, x)  # x <= alt'ın tepesi
    else:
        ekle(ust, x)
    if len(alt.a) > len(ust.a) + 1:  # dengele
        ekle(ust, cikar(alt))
    elif len(ust.a) > len(alt.a):
        ekle(alt, cikar(ust))


def ortanca(alt, ust):
    if len(alt.a) > len(ust.a):
        return float(alt.a[0])
    return (alt.a[0] + ust.a[0]) / 2.0
```

DİL PENCERESİ · **Farklılaşan satırlar**

Öbek C'de `struct` (işaretçi + boyut + tür bayrağı) ve `malloc`la ayrılmış dizidir; C++'ta `std::vector` taşıyan bir `struct`, Python'da küçük bir sınıftır (`class Obek`). En büyük/en küçük öbek ayrımı üç dilde aynı `en_buyuk` bayrağıyla karşılaştırma fonksiyonunda yapılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 21, 37, 126; Python 14, 15, 100.)